Є. В. Соколова, О. В. Лучшева, Ю. С. Манжос

ПРОГРАМУВАННЯ МОВОЮ C#

2021

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний аерокосмічний університет ім. М. Є. Жуковського
«Харківський авіаційний інститут»

Є. В. Соколова, О. В. Лучшева, Ю. С. Манжос

ПРОГРАМУВАННЯ МОВОЮ C#

Навчальний посібник до лабораторних робіт

Харків «ХАІ» 2021

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Національний аерокосмічний університет ім. М. Є. Жуковського
«Харківський авіаційний інститут»

Є. В. Соколова, О. В. Лучшева, Ю. С. Манжос

ПРОГРАМУВАННЯ МОВОЮ C#

Навчальний посібник до лабораторних робіт

Харків «ХАІ» 2021

УДК 004.89
С59

Рецензенти: канд. техн. наук, доц. В. М. Федорченко,
канд. техн. наук, доц. І. М. Іванісенко

Соколова, Є. В.

С59 Програмування мовою C# [Електронний ресурс] : навч. посіб. до лаб. робіт. / Є. В. Соколова, О. В. Лучшева, Ю. С. Манжос. – Харків : Нац. аерокосм. ун-т ім. М. Є. Жуковського «Харків. авіац. ін-т», 2021. – 75 с.

Навчальний посібник є продовженням першої частини і містить методики виконання розрахункової та лабораторних робіт з використанням середовища розробника MS Visual Studio мовою програмування C#.

Подано опис інструментів середовища програмування, розглянуто компоненти, властивості, методи і події, які використовують для розроблення програмного забезпечення C#. Викладено теоретичні відомості щодо деяких компонентів середовища.

Для студентів спеціальності «Інженерія програмного забезпечення» всіх форм навчання, а також для програмістів-початківців.

Іл. 17. Табл. 17. Бібліогр.: 7 назв

УДК 004.89

© Соколова Є. В., Лучшева О. В., Манжос Ю. С., 2021
© Національний аерокосмічний
університет ім. М. Є. Жуковського
«Харківський авіаційний інститут», 2021

ЗМІСТ

Лабораторна робота № 1. СТВОРЕННЯ ПРОСТИХ ФОРМ. ПРОГРАМУВАННЯ КНОПОК 4

Лабораторна робота № 2. ОБРОБЛЕННЯ ВИНЯТКОВИХ СИТУАЦІЙ 12

Лабораторна робота № 3. ОБРОБЛЕННЯ ОДНОВИМІРНИХ МАСИВІВ. КОМПОНЕНТ LISTBOX 20

Лабораторна робота № 4. ОБРОБЛЕННЯ ДИНАМІЧНИХ МАСИВІВ ДАНИХ. КОМПОНЕНТ DATAGRIDVIEW (ТАБЛИЦЯ РЯДКІВ) 27

Лабораторна робота № 5. ОБРОБЛЕННЯ РЯДКІВ. СТВОРЕННЯ МЕНЮ. ОБРОБЛЕННЯ ПОДІЙ КЛАВІАТУРИ 38

Лабораторна робота № 6. ТЕКСТОВІ ФАЙЛИ (ПОТОКИ СИМВОЛІВ) 45

Лабораторна робота № 7. ДВІЙКОВІ ФАЙЛИ (ПОТОКИ) 49

Лабораторна робота № 8. РОБОТА З ОБ’ЄКТАМИ. ІНКАПСУЛЯЦІЯ. УСПАДКУВАННЯ. ПОЛІМОРФІЗМ 56

Розрахункова робота. РЕКУРЕНТНЕ СПІВВІДНОШЕННЯ. ВИКОРИСТАННЯ РЕКУРСИВНИХ ФУНКЦІЙ 67

БІБЛІОГРАФІЧНИЙ СПИСОК 74

Лабораторна робота № 1

СТВОРЕННЯ ПРОСТИХ ФОРМ. ПРОГРАМУВАННЯ КНОПОК

Мета роботи – вивчення класів Form, Label, Button, TextBox та їх основних властивостей.

Теоретичні відомості

Призначення і склад вікон для візуального розроблення ПЗ у Microsoft Visual Studio

При створенні нового проєкту Windows Forms C# робочий стіл може містити декілька вікон, розташування і кількість яких залежить від налаштувань (рис. 1). Найбільш часто використовують такі:

- панель інструментів (Toolbox);
- оглядач рішень (Solution Explorer);
- вікно властивостей (Propeties);
- вікно виведення (Output);
- багатосторінкове вікно редактора.

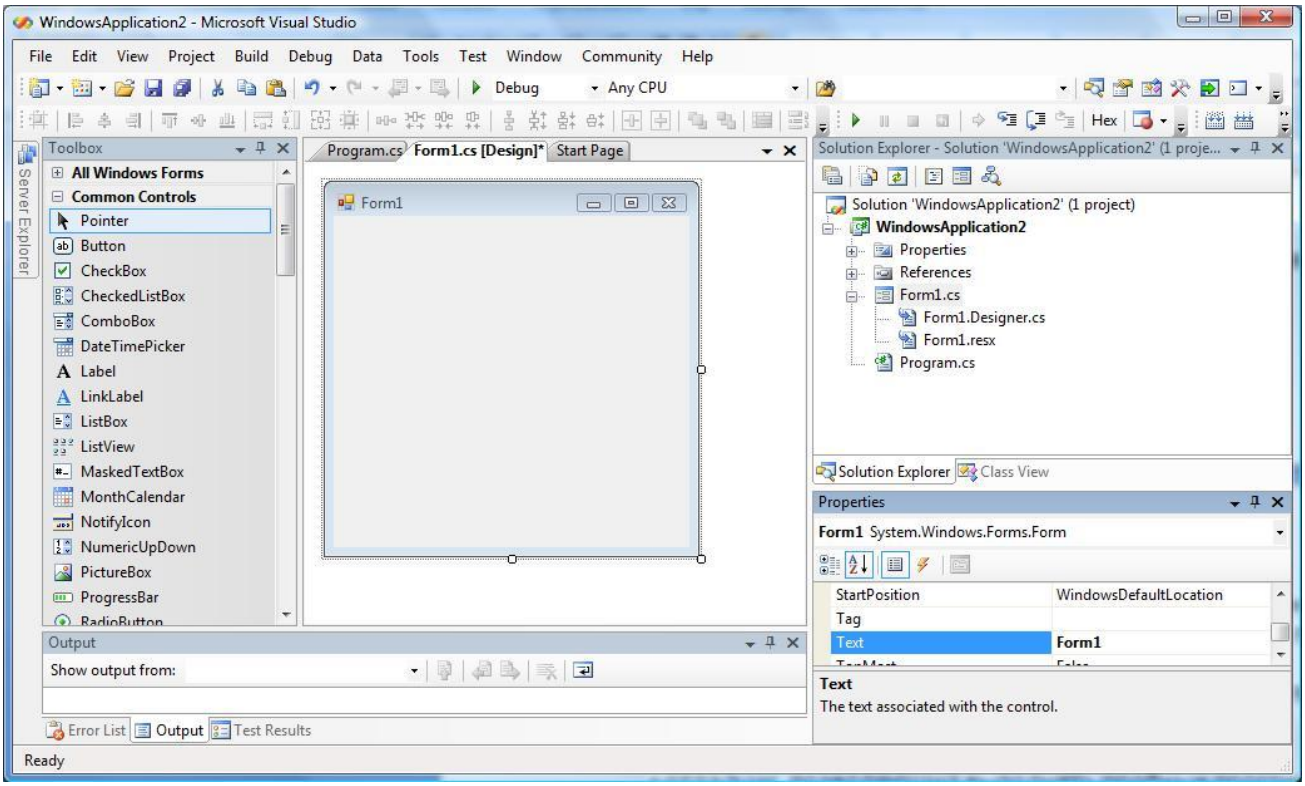


Рис. 1. Можливий вид екрана Microsoft Visual Studio при створенні C# Windows додатка

Клас Form (форма)

Клас ***Form*** використовують для створення нового об’єкта – вікна програми. Основні властивості форми наведено в табл. 1.

Таблиця 1

Основні властивості форми

Властивість	Опис властивості	Приклади значень
WindowState	Стан вікна в момент запуску програми	*Normal* – нормальний вид, *Minimized* – згорнутий, *Maximized* – розгорнутий на весь екран
ActiveControl	Задання активного об’єкта в формі	*button1, textBox1*
AutoScroll	Наявність смуги прокручування у формі	*True, False*
FormBoderStyle	Загальний вид вікна і операції з ним	*None* – без смуги заголовка, не допускає зміни розміру та переміщення; *FixedSingle* – вікно, розмір якого користувач не може змінити мишкою, а тільки клавішами в смузі заголовка; *Fixed3D* – має об’ємний вигляд; *FixedDialog* – вікно з фіксованими розмірами; *Sizable* – вікно з довільними розмірами і можливістю їх змінювати під час виконання; *FixedToolWindow* – те ж, що і *FixedSingle*, але зі смугою заголовка меншого розміру; *SizeableToolWin* – те ж, що і *Sizeable*, але зі смугою заголовка меншого розміру і без клавіш зміни розміру
Size *(With, Height)*	Ширина і висота вікна в пікселях	*503*, *224* (числове значення)
Font	Шрифт	Визначення параметрів шрифту (комплексна властивість задається в діалоговому вікні)

Закінчення табл. 1

Властивість	Опис властивості	Приклади значень
Icon	Визначення піктограми заголовка форми під час виконання програми	*None* – стандартна піктограма Microsoft Visual Studio або завантажена з будь-якого файла **.ico*
Name	Ім’я форми	*Form1* – ім’я форми (ідентифікатор для опису в програмі)
Text	Заголовок форми	Довільний рядок символів
BackColor	Колір фону форми	*Control, Yellow*
Cursor	Вид курсора миші при виконанні проєкту	*Hand, Arrow*
Enabled	Доступність компонентів під час виконання додатка	*True*, *False*
Left *Top*	Координати лівого верхнього кута вікна в пікселях	*200*, *108* (числове значення)
StartPosition	Розміщення і розміри вікна під час запуску програми	*CenterScreen* – у центрі екрана, *Manual* – за визначенням розробника

Клас Label (мітка)

Клас *Label* використовують для створення міток (написів) у вікні програми. Мітку поміщають у форму за допомогою кнопки A панелі інструментів. Крім властивостей, аналогічних наведеним у попередній таблиці (*Font*, *Name*, *Text*, *Cursor*, *Enabled)*, клас має властивості, зазначені в табл. 2.

Таблиця 2

Властивості класу Label

Властивість	Опис властивості	Приклади значень
TextAlign	Вирівнювання тексту відносно меж поля	TopCenter, MiddleRight
AutoSize	Зведення меж поля до меж тексту	True, False
Visible	Видимість об'єкта	True, False

Клас Button (кнопка)

Клас *Button* використовують для створення кнопки на формі, яку поміщають у форму за допомогою кнопки panelі інструментів.

Крім властивостей, аналогічних наведеним раніше (`Visible`, `With`, `Height`, `Font`, `Name`, `Text`, `Cursor`, `Enabled)`, кнопки мають властивості, зазначені в табл. 3.

Таблиця 3

Властивості класу Button

Властивість	Опис властивості	Приклади значень
ForeColor	Основний колір для відображення тексту	True, False
DialogResult	Результат діалогового вікна відображається в модальній формі після натискання клавіші	OK, Cancel

Для програмування кнопки *Button1* необхідно двічі натиснути на неї лівою клавішею миші, у наслідок чого активізується вікно тексту програми із заготовкою, яка буде обробляти подію, задану натисканням кнопки *Button1*:

```
private void button1_Click(object sender, EventArgs e)
{
. . .
}
```

У заготовку необхідно вставити текст програми, що містить реакцію на цю подію.

Клас TextBox (текстовий елемент керування)

Клас *TextBox* використовують для введення/виведення користувачем ряду символів з клавіатури.

Поле редагування поміщають на форму за допомогою кнопки на стандартній панелі інструментів.

За необхідністю перетворення отриманого рядка (властивість `Text`) у число і навпаки застосовують функції перетворення.

Крім відомих текстові елементи керування мають властивості, наведені в табл. 4.

Таблиця 4

Властивості класа TextBox

Властивість	Опис властивості	Приклади значень
CharacterCasing	Вид символів, які набирають у полі	Normal – звичайні; Upper – прописні; LowerCase – рядкові
PasswordChar	Символ для введення пароля	#0 – пряме відображення тексту; * – відображення тексту зірочками; 0 – відображення тексту нулями
ReadOnly	Можливість змінювати текст (доступність поля)	True – текст не можна змінити; False – текст можна змінити
Text	Текст у полі редагування	«89,3» – довільний ряд символів

Перетворення рядків у числа і навпаки

Оскільки багато об'єктів (типів *Label*, *TextBox* та ін.) сприймають інформацію тільки в рядковому вигляді (через властивість `Text`), то виникає задача перетворення рядка на різні за типом дані і навпаки – даних на рядок.

Рядки перетворюються на дані певного типу за допомогою методу Parse простору імен System.

Наприклад:

- рядок `s` перетворюється на еквівалентне ціле 32-розрядне знакове число типу `int`:

```
int x = int.Parse(s);
```

- рядок `s` перетворюється на еквівалентну кількість подвійної точності з плаваючою крапкою:

```
double x = double.Parse(s);
```

Для перетворення даних певного типу на рядок у просторі імен *System* є метод *ToString*, який перетворює значення об'єкта на еквівалентний йому рядок.

Наприклад:

- значення об'єкта `res` перетворюється на еквівалентне йому рядкове подання:

```
string s= res.ToString();
```

Метод установлення фокуса

Для встановлення фокуса на будь-який компонент форми використовують стандартний метод *SetFocus*.

Приклад 1. При передаванні фокуса від однієї кнопки до іншої використовують такий фрагмент програми:

```
private void button1_Click(object sender, EventArgs e)
        {
            button2.Focus();
        }

private void button2_Click(object sender, EventArgs e)
        {
            button1.Focus();
         }
```

Порядок виконання роботи

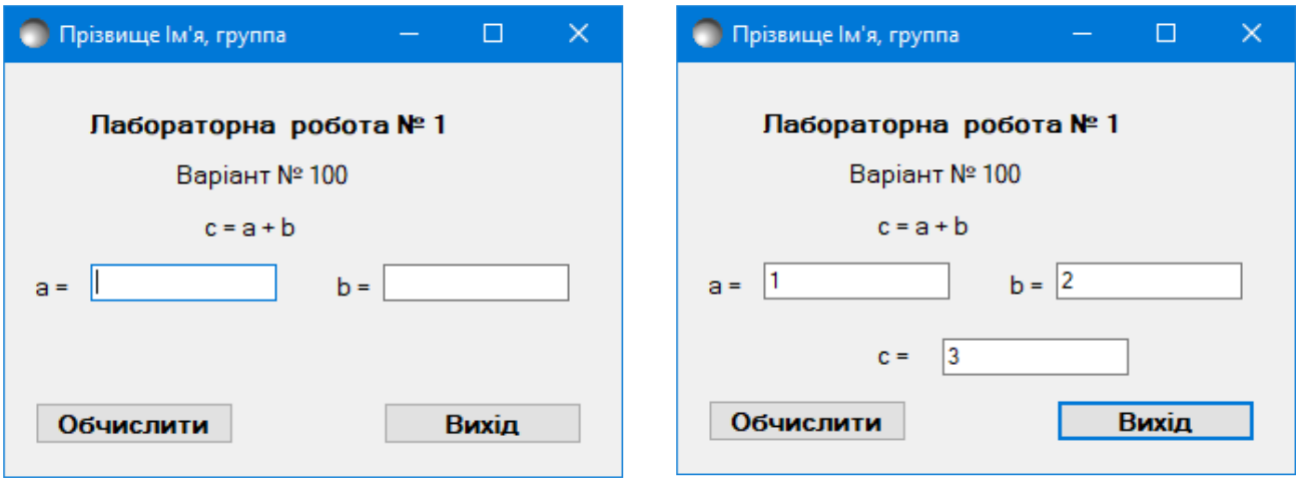
1. Створити форму, в заголовку якої вказати своє прізвище, ім'я, номер групи. Колір фону форми і шрифти всіх об'єктів підібрати самостійно, обов'язково змінивши установки за замовчуванням.
2. Розмістити по центру форми:
   - текстові мітки із зазначенням номера лабораторної роботи, номера варіанта і завдання;
   - текстові елементи керування з мітками для введення вихідних даних і виведення результату. Елемент керування для результату і його мітку зробити невидимими для його отримання;
   - кнопки «Обчислити» і «Вихід».

Усім об'єктам на формі дати смислові імена (наприклад, кнопка «Обчислити» – *btnCalc*, мітка з текстом «a =» – *lblA*, поле редагування для введення – *txtBxA*).

3. Написати обробники подій натискання на кнопки (*Click*). При натисканні на кнопку «Обчислити» обчислити функцію згідно з варіантом (табл. 5) і вивести результат на екран. При обробленні події обов'язково використовувати локальні змінні різних типів (типи підібрати самостійно) для вихідних даних і результату. При натисканні на кнопку «Вихід» закрити додаток.

4. Оформити звіт.

Форма має вигляд, показаний на рис. 2.



а б

Рис. 2. Вигляд форми лабораторної роботи № 1: а – до виконання обчислень; б – після виконання обчислень

Варіанти завдань до лабораторної роботи № 1

Таблиця 5

Функції для обчислення

Номер варіанта	Арифметичний вираз	Номер варіанта	Арифметичний вираз
1	$2c - d$	2	$c + 4d$
3	$2c + 82d$	4	$2c - a$
5	$a^2 - b$	6	$-2c - d + 53$

Закінчення табл. 5

Номер варіанта	Арифметичний вираз	Номер варіанта	Арифметичний вираз
7	$a^2 - c$	8	$c - 23d$
9	$2c + d$	10	$4c + d - 1$
11	$c + a - 1$	12	$d + a - 1$
13	$a^2 - b^2$	14	$a^2 + b$
15	$4d - a$	16	$c - a + 1$
17	$d - a + b$	18	$2c + d$
19	$a^2 - b - 5$	20	$d + a - 1$
21	$a^2 - b + 73$	22	$da + 1$
23	$b + ac$	24	$cd - 1$
25	$1 + cd$	26	$dc - 5$
27	$b + 1 - c$	28	$cd + 18$
29	$12 + a - b$	30	$ad - 1$

Контрольні запитання

1. Як створити форму? Перелічіть основні властивості форми.
2. Назвіть основні властивості об’єкта типа *Label*.
3. Укажіть основні властивості об’єкта типа *Button*.
4. Назвіть основні властивості об’єкта типа *TextBox*.
5. Дати визначення поняттю «подія». Укажіть події, які використовують для роботи з елементами керування.
6. Як встановити фокус на об’єкт?
7. Назвіть особливості введення/виведення даних різного типу на форму.

Лабораторна робота № 2

ОБРОБЛЕННЯ ВИНЯТКОВИХ СИТУАЦІЙ

Мета роботи – вивчення класу RadioButton і його основних властивостей. Оброблення виняткових ситуацій. Виведення діалогових вікон.

Теоретичні відомості

Клас RadioButton (перемикач)

Клас *RadioButton* дозволяє користувачеві вибрати один із запропонованих варіантів. Коли користувач установлює один з перемикачів, то всі інші автоматично знімаються. Перемикач поміщають на форму за допомогою кнопки панелі інструментів. Деякі властивості перемикачів наведено в табл. 6.

Таблиця 6

Властивості класа RadioButton

Властивість	Опис властивості	Приклади значень
Checked	Стан перемикача	True – увімкнений, False – вимкнений
Text	Текст, розташований справа від перемикача	«до 20 років» – довільний ряд символів

Приклад 2. При натисканні клавіші «Відповідь» у вікні з'являється рядок, який відповідає вибраному перемикачу (рис. 3).

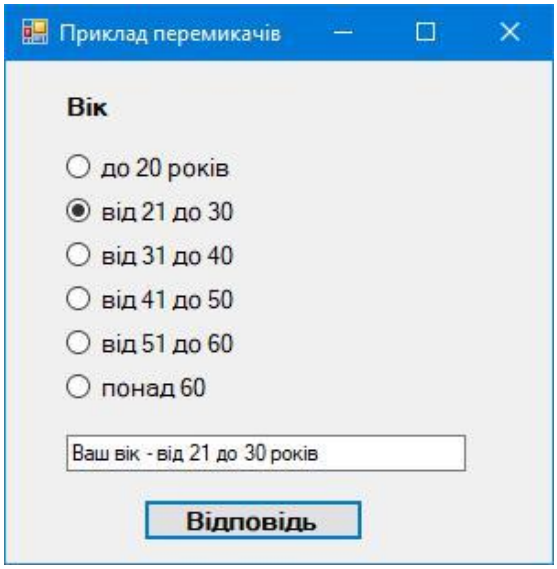


Рис. 3. Вигляд форми прикладу 2

Оброблення події, викликаної натисканням кнопки, має такий вигляд:

```
private void button1_Click(object sender, EventArgs e)
        {
            string s = "Ваш вік - ";
            if (radioButton1.Checked)
                s += "до 20 років"; if
            (radioButton2.Checked)
                s += "від 21 до 30 років";
            if (radioButton3.Checked)
                s += "від 31 до 40 років";
            if (radioButton4.Checked)
                s += "від 41 до 50 років";
            if (radioButton5.Checked)
                s += "від 51 до 60 років";
            if (radioButton6.Checked)
                s += "понад 60 років";
            textBox1.Text = s;
        }
```

Клас MessageBox для відображення вікон повідомлень

Вікно повідомлення – це готове модальне діалогове вікно, яке відображає користувачеві текстове повідомлення. Вікно повідомлення демонструється при виклику статичного методу *Show* класу *MessageBox*. Відображення текстового повідомлення – рядковий аргумент, який можна передати методу *Show*. Кілька перевантажених версій методу *Show* також дозволяють указати заголовок.

Наприклад:

- текстове вікно (рис. 4, а) виводить такий оператор:

```
MessageBox.Show("Приклад");
```

- текстове вікно (рис. 4, б) виводить такий оператор:

```
MessageBox.Show("Приклад","Заголовок");
```

а б

Рис. 4. Вигляд вікон повідомлень класу *MessageBox*

За допомогою вікна повідомлень можна також задати користувачеві запитання. Користувач відповідає натисканням однієї з кількох кнопок,

зазначених для відображення за допомогою параметра переліченого типу *MessageBoxButton*. Значення цього параметра можна передати кількома перевантаженими версіями методу *Show*, за замовчуванням значення якого є *OK*. Параметр типу *MessageBoxButton* може мати одне з наведених у табл. 7 значень.

Таблиця 7

Значення переліченого типу *MessageBoxButton*

Значення	Опис
OK	У вікні повідомлення відображається кнопка `OK`
OKCancel	У вікні повідомлення відображаються кнопки `OK` і `Cancel` *(Відмінити)*
YesNoCancel	У вікні повідомлення відображаються кнопки `Yes` *(Так)*, `No` *(Ні)*, `Cancel` *(Відмінити)*
YesNo	У вікні повідомлення відображаються кнопки `Yes` *(Так)*, `No` *(Ні)*
AbortRetryIgnore	У вікні повідомлення відображаються кнопки `Abort` *(Перервати)*, `Retry` *(Повторити)*, `Ignore` *(Пропустити)*
RetryCancel	У вікні повідомлення відображаються кнопки `Retry` *(Повторити)*, `Ignore` *(Пропустити)*

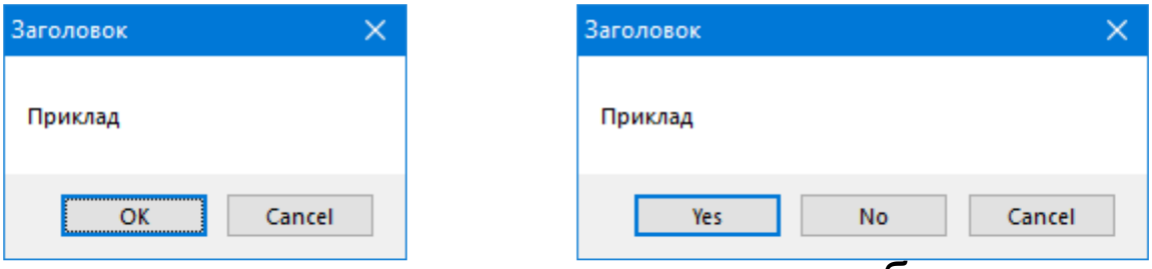
Наприклад:

– текстове вікно (рис. 5, а) виводить такий оператор:

```
MessageBox.Show("Приклад", "Заголовок", MessageBoxButtons.
OKCancel);
```

– текстове вікно (рис. 5, б) виводить такий оператор:

```
MessageBox.Show("Приклад", "Заголовок", MessageBoxButtons.
YesNoCancel);
```



а б

Рис. 5. Вигляд вікон повідомлення класу *MessageBox* з набором кнопок для відповіді

Можна визначити, яку кнопку натисне користувач, проаналізувавши значення результату роботи методу ***Show***. Це значення переліченого типу

DialogResult, де кожне значення належить одній із кнопок, що відображається у вікні повідомлення. Значення за замовчуванням для вікна повідомлення – *OK*, оскільки *OK* у вікні повідомлення – це кнопка за замовчуванням.

Наприклад, фрагмент кода, який показує аналіз натиснутої кнопки, має такий вигляд:

```
DialogResult res = MessageBox.Show("Приклад","Заголовок",
MessageBoxButtons.YesNo);
if (res == DialogResult.Yes)
    label1.Text = "Натиснули кнопку Yes";
if (res == DialogResult.No)
    label1.Text = "Натиснули кнопку No";
```

Вікна повідомлень можуть обмінюватися інформацією і ставити запитання різного ступеня важливості. Для позначення важливості у вікнах повідомлень використовують значки. Наприклад, значки можуть означати, чи є повідомлення інформаційними, попереджувальними або важливими. Перелічений тип ***MessageBoxImage*** містить набір можливих значень для формування зображень у вікнах повідомлень. За замовчуванням вікно повідомлення не відображає знак. Однак можна передати значення ***MessageBoxImage*** однією з декількох версій методу ***Show***, щоб указати, що вікно має значок. ***MessageBoxImage*** може набувати значеннь, наведених у табл. 8.

Таблиця 8

Значення переліченого типу *MessageBoxImage*

Значення	Опис
None	Знак не відображається
Hand	У вікні повідомлення відображається `знак руки`
Question	У вікні повідомлення відображається `знак запитання`
Exclamation	У вікні повідомлення відображається `знак оклику`
Asterisk	У вікні повідомлення відображається `знак зірочки`
Stop	У вікні повідомлення відображається `знак зупинки`
Error	У вікні повідомлення відображається `знак помилки`
Warning	У вікні повідомлення відображається `знак попередження`
Information	У вікні повідомлення відображається `знак даних`

Наприклад:

– текстове вікно (рис. 6, а) виводить такий оператор:

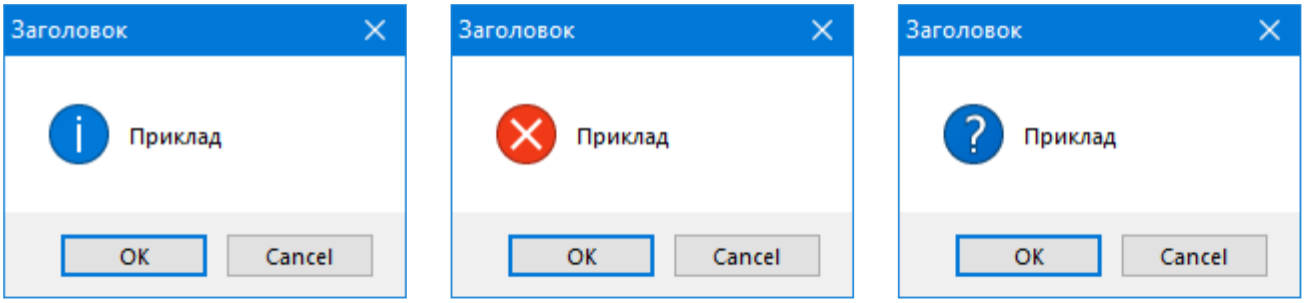
```
MessageBox.Show("Приклад","Заголовок",MessageBoxButtons.
OKCancel,MessageBoxIcon.Information);
```

– текстове вікно (рис. 6, б) виводить такий оператор:

```
MessageBox.Show("Приклад","Заголовок",MessageBoxButtons.
OKCancel,MessageBoxIcon.Stop);
```

– текстове вікно (рис. 6, в) виводить такий оператор:

```
MessageBox.Show("Приклад","Заголовок",MessageBoxButtons.
OKCancel,MessageBoxIcon.Question);
```



а б в

Рис. 6. Вигляд вікон повідомлень класу *MessageBox* зі знаками і набором кнопок для відповіді

Оброблення виняткових ситуацій

В C# передбачено засоби перехоплення помилок, що виникають у вигляді винятків при виконанні програми. Виключення генерується, коли виникає помилка або інша подія, що порушує нормальну роботу програми. Згенерований виняток передає керування обробнику винятків, тобто фрагменту кода, який виконується при появі помилки. Синтаксис операторів захисту від помилок має такий вигляд:

```
try
     {Захищені оператори;}
catch (тип_винятку 1 <ім'я_винятку 1>)
     {оператори оброблення винятку 1;}
finally
    {оператори, які завжди виконуються;}
```

Виконується блок пропозиції в пропозиції `try`. Якщо при виконанні рядків блока або методів, які викликаються ними, виникає помилка типу `тип_винятку 1`, то керування передається на блок `оператори оброблення винятку 1` в обробнику `catch`. У програмі може бути будь-

яка кількість пропозицій `catch`; з них виконується перша, що відповідає за помилку. Якщо `<ім'я_винятку 1>` не використовується у наступних пропозиціях, його можна опустити.

При виникненні винятку виконання процесів зупиняється і керування передається найближчому обробнику винятків. Часто це означає, що не виконуються ті рядки коду, які завжди викликаються. Очищення деяких ресурсів має виконуватися завжди, навіть якщо створюється виняток. Для цього можна використовувати блок `finally`, оператори якого виконуються завжди незалежно від виникнення винятку.

Винятки можна створювати явно за допомогою оператора `throw`. Синтаксис винятку:

```
throw new <тип_винятку>("повідомлення");
```

Порядок виконання роботи

1. Створити форму і розмістити на формі:

– текстові мітки із зазначенням номера лабораторної роботи, номера варіанта і постановки завдання;

– текстові елементи керування з мітками для введення вихідних даних і виведення результату. Елемент керування для результату і його мітку зробити невидимими для його отримання;

– кнопки «Обчислити» і «Вихід». При натисканні кнопки «Обчислити» має обчислюватися вираз відповідно до варіанта (табл. 9) з виведенням результату у відповідний елемент керування, який не має бути доступним для введення (доступним тільки для читання). При натисканні кнопки «Вихід» вікно програми закривається;

– перемикачі, які визначають, контролювати виняткові ситуації чи ні.

2. При виконанні оброблення виняткових ситуацій вивести відповідні діалогові вікна. Можливий вигляд головної форми та діалогових вікон показано на рис. 7.

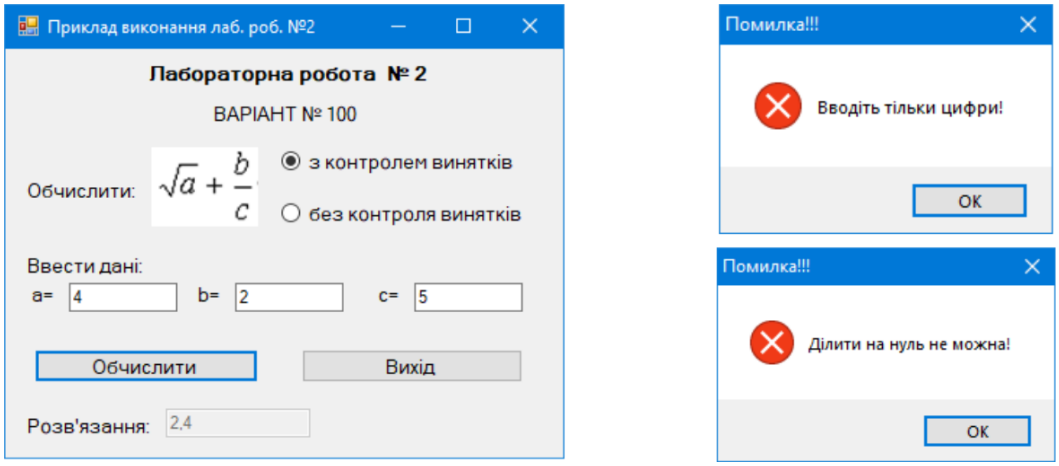


Рис. 7. Екранні форми лабораторної роботи № 2

Варіанти завдань до лабораторної роботи № 2

Таблиця 9

Функції для обчислення

Номер варіанта	Арифметичний вираз	Номер варіанта	Арифметичний вираз
1	$\frac{\mathrm{a}+\frac{c}{b}-\sqrt{28d}}{4\mathrm{ab}+1}$	2	$\frac{\frac{c}{b}-\sqrt{24+d}+a}{2\mathrm{ac}-1}$
3	$\frac{2\mathrm{b}-\ln(\mathrm{a}+\mathrm{b})\mathrm{c}}{\frac{c}{4}-\mathrm{d}}$	4	$\frac{41-\frac{d}{4}-1}{\frac{tg(b+a)}{c}-\mathrm{d}}$
5	$\frac{\mathrm{a}-4\mathrm{b}-10}{\frac{31}{c}+\mathrm{tg}(\mathrm{ab})}$	6	$\frac{\lg\left(\frac{b}{a}\right)+4+\mathrm{d}}{\mathrm{c}-\mathrm{b}+1}$
7	$\frac{\lg(21-\mathrm{a})\frac{c}{2}}{1+\frac{c}{a}+\mathrm{b}}$	8	$\frac{c-\frac{\ln(33+b)}{4}}{a-\frac{c}{b}-1}$
9	$\frac{2\mathrm{b}-\frac{3}{c}}{\frac{arctg(b+a)}{c}+1}$	10	$\frac{arctg\left(\frac{c}{4}+28\right)\mathrm{d}}{\frac{a}{d}-\mathrm{c}-1}$
11	$\frac{\frac{ab}{4}-1}{\sqrt{41-d}-ab+c}$	12	$\frac{1+a-\frac{b}{2}}{\sqrt{24+d-c}+\frac{a}{b}}$
13	$\frac{\ln(\mathrm{ab}+2)\mathrm{c}}{41-\frac{b}{c}+\mathrm{d}}$	14	$\frac{lg(4b-c)\mathrm{a}}{b+\frac{c}{d}-1}$
15	$\frac{2\mathrm{c}+\frac{\mathrm{tg}(\mathrm{a}-21)}{b}}{\frac{c}{a}+\mathrm{d}+1}$	16	$\frac{\frac{4}{c}+tg(3a)}{\frac{c}{a}-\mathrm{b}-1}$
17	$\frac{8\lg(\mathrm{b}-1)-\mathrm{c}}{2a+\frac{b}{c}}$	18	$\frac{4ln\left(\frac{a}{b}\right)+1}{c+b-d+a}$
19	$\frac{5ln\left(\frac{b}{c+1}\right)+1}{2c+a-b}$	20	$\frac{\frac{arctg(b-c)}{b}+\frac{a}{4}}{a+b-1}$

Закінчення табл. 9

Номер варіанта	Арифметичний вираз	Номер варіанта	Арифметичний вираз
21	$\dfrac{arctg\left(\dfrac{a-c}{c+1}\right)b+28}{4\dfrac{b}{a}+1}$	22	$\dfrac{bc-24+a}{b-\lg(2c-1)+a}$
23	$\dfrac{\sqrt{2b}-a+\dfrac{b}{c}}{a-\dfrac{c}{4}+1}$	24	$\dfrac{\sqrt{\dfrac{41d}{a}}+1}{a-\dfrac{c}{b}+d}$
25	$\dfrac{a+tg\left(\dfrac{b}{4}-1\right)+\dfrac{b}{c}}{\dfrac{c}{3}-ad}$	26	$\dfrac{ab+\dfrac{2}{c}}{4c-tg(b+1)}$
27	$\dfrac{\lg\left(25+\dfrac{2a}{d}\right)}{c+a-b-1}$	28	$\dfrac{c+23-4b}{a+ln\left(a+\dfrac{c}{b}-1\right)}$
29	$\dfrac{\dfrac{2}{b}-\dfrac{53}{c}}{arctg(b-a)c+1}$	30	$\dfrac{4c+28\dfrac{d}{c}}{5-arctg(ad-c-1)}$

Контрольні запитання

1. Назвіть основні властивості компонента RadioButton.
2. Як використовують клас MessageBox для виведення повідомлень?
3. Як застосовують метод Show для виведення різного роду повідомлень?
4. Як виконується оброблення виняткових ситуацій?

Лабораторна робота № 3

ОБРОБЛЕННЯ ОДНОВИМІРНИХ МАСИВІВ. КОМПОНЕНТ LISTBOX

Мета роботи – вивчення класу ListBox і його основних властивостей. Оброблення одновимірних масивів за допомогою класу Array.

Теоретичні відомості

Одновимірні масиви

Одновимірний масив – це список пов'язаних змінних. Для об’явлення одновимірного масиву використовують такі різні форми запису:

```
тип[] ім’я_масиву;
тип[] ім’я_масиву = new тип[розмір];
тип[] ім’я_масиву = {список ініціалізаторів};
тип[] ім’я_масиву = new тип[] {список ініціалізаторів};
тип[] ім’я_масиву = new тип[розмір] {список
ініціалізаторів};
```

У мові програмування C # масиви реалізовані як об'єкти класу Array, що надає методи для створення, зміни, пошуку і сортування масивів, тобто являє собою базовий клас для всіх масивів у середовищі CLR. Деякі найбільш часто використовувані властивості та методи наведено в табл. 10.

Таблиця 10

Властивості і методи класу Array

Властивість/метод	Опис
Length	Властивість, що визначає довжину вектора
Clear	Метод, що очищує масив і установлює для всіх його елементів значення за замовчуванням
Resize	Метод, що замінює розмір масиву вказаним новим розміром
Copy	Метод, що копіює частину одного масиву в інший
Reverse	Метод, що розташовує елементи масиву в зворотному порядку
Sort	Метод, що сортує елементи одновимірного масиву

Клас ListBox (відображення списку)

Елемент керування *ListBox* дозволяє відобразити список позицій, з яких користувач може вибрати потрібну натисканням миші. Деякі властивості списку наведено в табл. 11.

Таблиця 11

Властивості класу ListBox

Властивість	Опис властивості
Items	Отримує позиції елемента списку
Multicolumn	Визначає, чи підтримує елемент керування ***ListBox*** декілька стовпців. Наприклад: `True` – підтримує, `False` – не підтримує (за замовчуванням)
SelectedIndex	Отримує або задає індекс виділеної в певний момент позиції в елементі керування ***ListBox*** (індекс починається з нуля)
SelectedIndices	Отримує колекцію, що містить індекси всіх виділених у даний момент позицій в елементі керування ***ListBox*** (індекс починається з нуля)
SelectedItem	Отримує або задає вибраний в певний момент елемент у полі зі списком ***ListBox***
SelectedItems	Отримує колекцію, яка містить виділені в певний момент позиції елемента керування ***ListBox***
SelectionMode	Отримує або задає режим вибору в елементі керування ***ListBox.*** Наприклад: `One` – дозволяє вибирати лише один елемент (за замовчуванням); `None` – не дозволяє вибирати; `MultiSimple`, `MultiExtended` – дозволяє вибирати декілька значень

Приклад 3. При натисканні кнопки «Формування вектора» створюється вектор випадкової довжини і відображається в компоненті *ListBox*. При натисканні кнопки «Вибране значення» виділені в списку елементи вектора відображаються в багаторядковому *TextBox* (рис. 8).

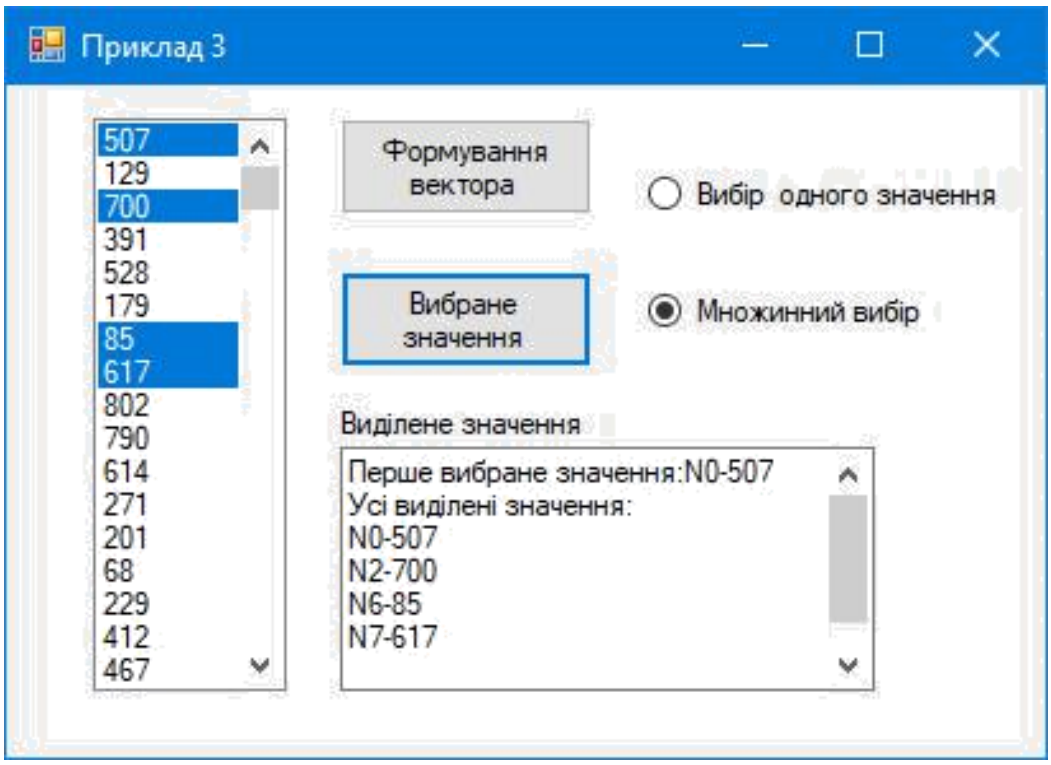


Рис. 8. Вигляд форми прикладу 3

Оброблення події, викликаної натисканням кнопки, має такий вигляд:

```
int []A=new int[]{};
private void button1_Click(object sender, EventArgs e)
    {
      Random m = new Random();
      int i=m.Next(1000);
      MessageBox.Show("Буде створений вектор розміру " +
i.ToString());
      form_vector(ref A, i);
      output_vector(A, listBox1);
    }
//формування вектора за допомогою датчика випадкових чисел
static void form_vector(ref int[] vec, int size)
    {
      Array.Resize(ref vec, size);
      Random m = new Random();
      for (int k = 0; k < vec.Length; k++)
        vec[k] = m.Next(size);
    }
//виведення вектора в компонент ListBox
static void output_vector(int[] vec, ListBox lstbxMy)
    {
      lstbxMy.Items.Clear();
      foreach (int el in vec)
        lstbxMy.Items.Add(el);
    }
```

Порядок виконання роботи

1. Створити форму і розмістити:
   - текстові мітки із зазначенням номера варіанта і постановки завдання;
   - текстові елементи керування з мітками для введення даних і виведення результату. Елемент керування і мітку для результату зробити невидимими до його отримання;
   - списки значень, в які виводити результати;
   - кнопки «Формування вектора» і «Отримання результату». При натисканні кнопки «Формування вектора» створювати вектор, що містить випадкові числа в діапазоні [-n, n], де n – кількість елементів вектора, і виводити елементи вектора у відповідний ListBox. При натисканні кнопки «Отримання результату» обчислити значення згідно з варіантом і вивести його на форму, а також змінити початковий вектор відповідно до варіанта і вивести змінений вектор у відповідний список.
2. Кнопка «Отримання результату» має стати доступною тільки тоді, коли початковий вектор сформований, і знову стати недоступною після отримання результуючого вектора.
3. Програма має обов'язково містити такі методи:
   - формування вектора за допомогою датчика випадкових чисел;
   - виведення вектора на форму в елемент керування *ListBox*;
   - розрахунок значення;
   - зміни вектора.
4. Передбачити оброблення можливих виняткових ситуацій.

Можливий вигляд головної форми показано на рис. 9.

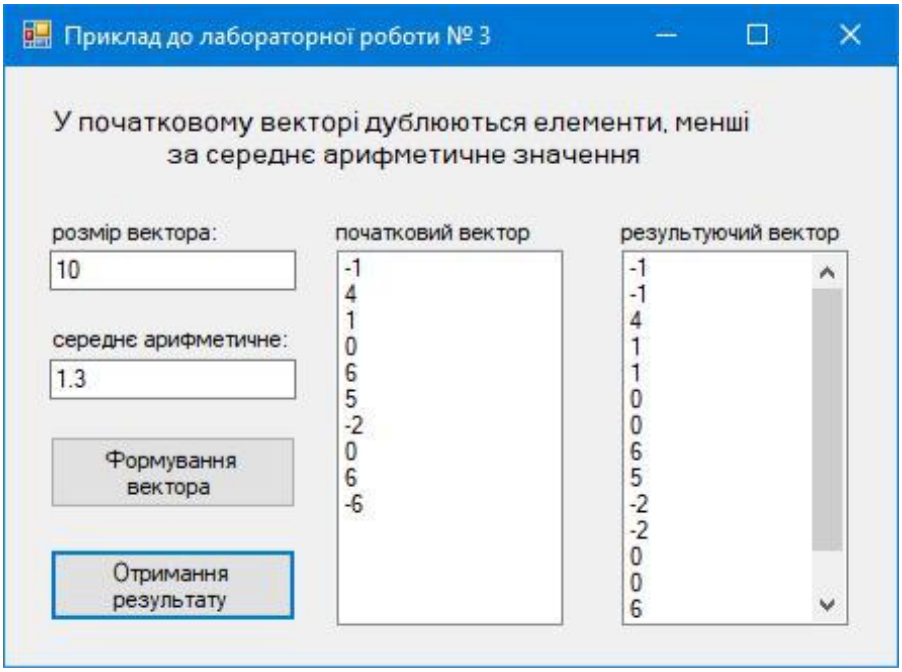


Рис. 9. Екранна форма лабораторної роботи № 3

Варіанти завдань до лабораторної роботи № 3

1. Знайти кількість непарних чисел серед елементів масиву A = {a[i]}. Додати в середину початкового вектора випадкову (від 1 до 10) кількість елементів. Доданим елементам присвоїти значення, що дорівнюють їх індексам.

2. Знайти добуток парних елементів масиву A = {a[i]}. Видалити із середини початкового вектора випадкову (від 1 до 10) кількість елементів.

3. Знайти суму всіх елементів масиву A = {a[i]}, що мають парні порядкові номери і є непарними числами. Додати в початок вектора випадкову (від 1 до 10) кількість елементів. Доданим елементам присвоїти нульові значення.

4. Знайти кількість елементів масиву A = {a[i]}, менших середнього арифметичного. Видалити з початкового вектора всі елементи, менші середнього арифметичного.

5. Знайти добуток усіх від'ємних елементів масиву A = {a[i]}. Продублювати всі від’ємні елементи початкового вектора.

6. Знайти, скільки елементів масиву A={a[i]} є парними від’ємними числами. Видалити з початку вектора кількість елементів, яка дорівнює кількості парних від’ємних чисел.

7. Знайти добуток додатних непарних елементів масиву A = {a[i]}. Додати після першого додатного непарного елемента випадкову (від 1 до 10) кількість елементів. Доданим елементам присвоїти значення, протилежне їх індексам.

8. Знайти суму всіх елементів масиву A = {a[i]}, що мають непарні порядкові номери і є непарними числами. Видалити після останнього непарного елемента з непарним порядковим номером випадкову (від 1 до 10) кількість елементів.

9. Знайти добуток усіх елементів масиву A = {a[i]}, більших середнього арифметичного. Потроїти всі елементи, більші середнього арифметичного.

10. Знайти суму від’ємних елементів масиву A = {a[i]}. Видалити всі від’ємні елементи вектора.

11. Знайти кількість подвоєних непарних елементів масиву A = {a[i]}. Додати в середину вектора випадкову (від 1 до 10) кількість елементів. Доданим елементам присвоїти значення, яке дорівнює кількості подвоєних непарних елементів.

12. Знайти кількість повних квадратів (число є квадратом цілого числа, наприклад, 64 – повний квадрат восьми) серед елементів масиву A = {a[i]}. Видалити після першого повного квадрата випадкову (від 1 до 10) кількість елементів.

13. Знайти довжину найменшого відрізка числової осі, що містить усі елементи масиву A = {a[i]}. Додати в початок вектора випадкову (від 1 до 10) кількість елементів. Доданим елементам присвоїти отримане значення довжини найменшого відрізка.

14. Знайти добуток від'ємних парних елементів масиву A = {a[i]}. Видалити після першого від'ємного парного елемента масиву випадкову (від 1 до 10) кількість елементів.

15. Знайти добуток додатних елементів масиву A = {a[i]}. Подвоїти всі додатні елементи масиву.

16. Знайти кількість подвоєних непарних елементів масиву A = {a[i]}. Видалити всі подвоєні непарні елементи.

17. Знайти максимальне число серед повних квадратів (число є квадратом цілого числа, наприклад, 64 – повний квадрат восьми) елементів масиву A = {a[i]}. Додати після останнього повного квадрата випадкову (від 1 до 10) кількість елементів. Доданим елементам присвоїти нульові значення.

18. Знайти кількість повних квадратів (число є квадратом цілого числа, наприклад, 64 – повний квадрат восьми) серед непарних елементів масиву A = {a[i]}. Видалити елементи вектора, які стоять перед першим повним квадратом серед непарних елементів.

19. Знайти номер останнього непарного елемента масиву A = {a[i]}. Додати після останнього непарного елемента масиву випадкову (від 1 до 10) кількість елементів. Доданим елементам присвоїти індекс останнього непарного елемента.

20. Знайти середнє арифметичне від'ємних елементів масиву A = {a[i]}. Видалити всі додатні елементи вектора.

21. Знайти мінімальний непарний елемент масиву A = {a[i]}. Додати перед мінімальним непарним елементом випадкову (від 1 до 10) кількість елементів.

22. Знайти максимальний парний елемент масиву A = {a[i]}. Видалити всі елементи, які стоять перед максимальним парним елементом.

23. Знайти кількість елементів масиву A = {a[i]}, які є степенем двійки (1, 2, 4…). Потроїти всі елементи, які є степенем двійки.

24. Знайти кількість парних чисел у масиві A = {a[i]}. Видалити із середини вектора випадкову (від 1 до 10) кількість елементів.

25. Знайти суму індексів непарних елементів масиву A = {a[i]}. Додати в початок вектора випадкову (від 1 до 10) кількість елементів. Доданим елементам присвоїти знайдену суму індексів від'ємних елементів.

26. Знайти добуток від'ємних парних елементів масиву A = {a[i]}. Видалити всі від'ємні парні елементи.

27. Знайти суму подвоєних парних елементів масиву A = {a[i]}. Додати в початок вектора випадкову (від 1 до 10) кількість елементів. Доданим елементам присвоїти знайдену суму подвоєних парних елементів.

28. Знайти номер першого парного елемента масиву A = {a[i]}. Видалити всі елементи, які стоять перед першим парним елементом масиву.

29. Знайти кількість парних чисел серед елементів масиву A = {a[i]}. Додати в початок початкового вектора випадкову (від 1 до 10) кількість елементів. Доданим елементам присвоїти значення, протилежні їх індексам.

30. Знайти добуток непарних елементів масиву A = {a[i]}. Видалити з початку початкового вектора випадкову (від 1 до 10) кількість елементів.

Контрольні запитання

1. Назвіть основні властивості компонента *ListBox*.
2. Опишіть особливості роботи з одновимірними масивами в C#.
3. Як використовують оператор foreach при роботі з векторами?

Лабораторна робота № 4

ОБРОБЛЕННЯ ДИНАМІЧНИХ МАСИВІВ ДАНИХ. КОМПОНЕНТ DATAGRIDVIEW (ТАБЛИЦЯ РЯДКІВ)

Мета роботи – вивчення класу DataGridView і його основних властивостей. Оброблення двовимірних масивів.

Теоретичні відомості

Багатовимірні масиви

Крім простих масивів (масивів з одним виміром) у мові програмування C# підтримуються два основні різновиди багатовимірних масивів:

− прямокутні (rectangle). Такий тип масиву утворюється простим додаванням кількох вимірів. При цьому всі рядки і стовпці будуть однакової довжини.

Оголошення таких масивів має вигляд:

```
тип[,] ім’я_масиву;
тип[,] ім’я_масиву = new тип [розмір_1,розмір_2];
тип[,] ім’я_масиву = {список ініціалізаторів};
тип[,] ім’я_масиву = new тип [,] {список ініціалізаторів};
тип[,] ім’я_масиву = new тип [розмір_1,розмір_2] {список
ініціалізаторів};
```

Наприклад*:*

```
int [,] a;
//елементів немає; описано тільки посилання на масив
int [,] b=new int [2,3];
//елементи в масиві дорівнюють 0
int [,] c={{5,7},{-9,8,4}};
//виділено пам'ять під п’ять елементів
int [,] d=new int[,] {{5,7},{-9,8,4,-4}};
//розмірність обчислюється
int [,] e=new int[2,3] {{5,7,-9},{8,4,-4}};
//надлишковий опис, розмір має бути постійним.
```

Доступ до елементів масиву:

```
ім’я_масиву [ном_рядка, ном_стовпчика]
```

− ламані (ступінчасті, jagged**)**. Такий масив містить деяку кількість внутрішніх масивів у якості внутрішніх елементів, кожен з яких може мати свій внутрішній унікальний розмір.

Оголошення таких масивів має вигляд:

```
тип[][] ім’я_масиву;
```

Під кожен з масивів, що становить ламаний масив, пам'ять потрібно виділяти явно.

Наприклад:

```
int [][] a=new int [3][];
//пам'ять під три рядки
посилання a[0]=new int [5];
a[1]=new int [7];
a[2]=new int [2];
```

Доступ до елементів масиву:

```
ім'я_масиву [ном_рядка][ном_стовпчика]
```

Елемент керування DataGridView (Таблиця)

Елемент керування *DataGridView* (Таблиця) відображає дані в сітці, що настроюється. Додається компонент за допомогою кнопки групи *Data* (*Дані)* панелі інструментів.

Деякі властивості елемента керування *DataGridView* наведено в табл. 12.

Таблиця 12

Властивості класу *DataGridView*

Властивість	Опис властивості
Rows	Колекція всіх рядків
Colums	Колекція всіх стовпчиків
ColumnCount	Отримує або задає кількість стовпчиків у елементі
DataGridViewRowCells	Повертає колекцію комірок, що заповнюють рядок
Rows.Add(numb)	Добавляє numb рядків
Columns[i].ReadOnly	Встановлення властивості Readonly (тільки для читання) для стовпчика «i»
Columns[i].SortMode	Встановлення режиму сортування для стовпчика «i»
Columns[i].Name	Встановлення властивості «Name» для стовпчика «i» (заголовок стовпчика)
Columns[i].Width	Встановлення ширини стовпчика «i»
Rows[i].Cells[j].Value	Значення комірки, що знаходиться на перетині рядка «i» й стовпчика «j»
SelectedCells.Count	Кількість виділених комірок
SelectedCells	Колекція виділених комірок

Приклад 4. Створити і вивести на екран динамічну матрицю цілих чисел вказаного розміру. Матрицю заповнити за допомогою датчика випадкових чисел. Для матриці знайти максимальні елементи кожного стовпчика. Відобразити виділений елемент матриці. Екранні форми програми показано на рис. 10. Фрагмент кода програми з описом матриці, функцій формування і виведення матриці в елемент керування `DataGridView` має такий вигляд:

```
//описуємо покажчик на матрицю
int[][] A = new int[0][];

private void button1_Click(object sender, EventArgs e)
   {
    //кількість рядків матриці
    int size = int.Parse(textBox1.Text);
    //виділяємо пам'ять під рядки матриці
    Array.Resize(ref A, size);
    //формуємо матрицю
    form_matrix(ref A, size);
    //заповнюємо матрицю випадковими
    числами RandArray(A);
    //виводимо матрицю в
    DataGridView Output_Matrix(A,
    dataGridView1);
    //описуємо вектор максимальних значень
    стовпчиків int [] B = new int[0];
    //знаходимо максимальні значення
    стовпчиків max_column(A, ref B);
    //виводимо вектор у TextBox
    Output_vector(B, textBox2);
    button2.Visible = true;
   }


//виведення вектора в TextBox
static void Output_vector(int[] Vec, TextBox txtbxMy)
   {
    txtbxMy.Text = "";
    foreach (int el in Vec)
      txtbxMy.Text += el.ToString() + "\t";
   }
```

```
    //формування вектора максимальних елементів стовпчиків
    static void max_column(int[][] Matr, ref int[] Max_col)
       {
        for (int i = 0; i < Matr.Length; i++)
         {
          for (int j = 0; j < Matr[i].Length; j++)
           //перевірка існування стовпчика
           if (Max_col.Length-1<j)
           {
//виділення пам’яті та ініціалізація
//елемента вектора максимальних значень стовпчиків
          Array.Resize(ref Max_col, Max_col.Length + 1);
          Max_col[j] = Matr[i][j];
           }
            else
            if (Matr[i][j] > Max_col[j])
     // ініціалізацію здійснено порівнянням
                Max_col[j] = Matr[i][j];
        }
       }

    //формування матриці
    static void form_matrix(ref int[][] Matr, int row)
       {

        // ініціалізація датчика випадкових чисел
        Random m=new Random();
        for (int i = 0; i < Matr.Length; i++)
         {
          //кількість елементів у рядку
          int col = m.Next(Matr.Length);
          Matr[i]=new int[col];
         }
       }

    //заповнення матриці випадковими числами
    static void RandArray(int[][] Mas)
       {
        Random m = new Random();
        for (int i = 0; i < Mas.Length; i++)
         for (int j = 0; j < Mas[i].Length; j++)
           Mas[i][j] = m.Next(Mas.Length * Mas[i].Length);
       }
```

```
    //виведення матриці на екран у DataGridView
    static void Output_Matrix(int[][] Mas, DataGridView
dtgrdvw)
        {
    //знаходимо максимальну кількість стовпчиків у
        рядках int max_col = Mas[0].Length;
        for (int i = 0; i < Mas.Length; i++)
         if (Mas[i].Length > max_col)
            max_col = Mas[i].Length;
        dtgrdvw.ColumnCount =
        0;//очищення
    //кількість стовпчиків
        dtgrdvw.ColumnCount =
        max_col;
    //кількість рядків
        dtgrdvw.Rows.Add(Mas.Length
        );
    //значення для стовпчиків
        for (int i = 0; i < max_col; i++)
         {
    //забороняємо змінювати значення комірок
         dtgrdvw.Columns[i].ReadOnly = true;
    //вимикаємо режим сортування елементів стовпчика
         dtgrdvw.Columns[i].SortMode =
DataGridViewColumnSortMode.NotSortable;
    //заголовок стовпчиків
         dtgrdvw.Columns[i].Name = i.ToString();
         dtgrdvw.Columns[i].Width = 25;//ширина стовпчиків
         }
    //відображення значень елементів матриці
    //DataGridView
        for (int i = 0; i < Mas.Length; i++)
        for (int j = 0; j < Mas[i].Length; j++)
          //заповнення комірки
          dtgrdvw.Rows[i].Cells[j].Value = Mas[i][j].
ToString();
          dtgrdvw.Visible = true;
        }

    //відображення виділеного значення
    private void button2_Click(object sender, EventArgs e)
        {
        if (dataGridView1.SelectedCells[0].Value != null)
```

```
        {
         string val =
dataGridView1.SelectedCells[0].Value.ToString();
         MessageBox.Show("Виділено: " + val);
        }
       else
        MessageBox.Show("Порожня комірка");
     }
```

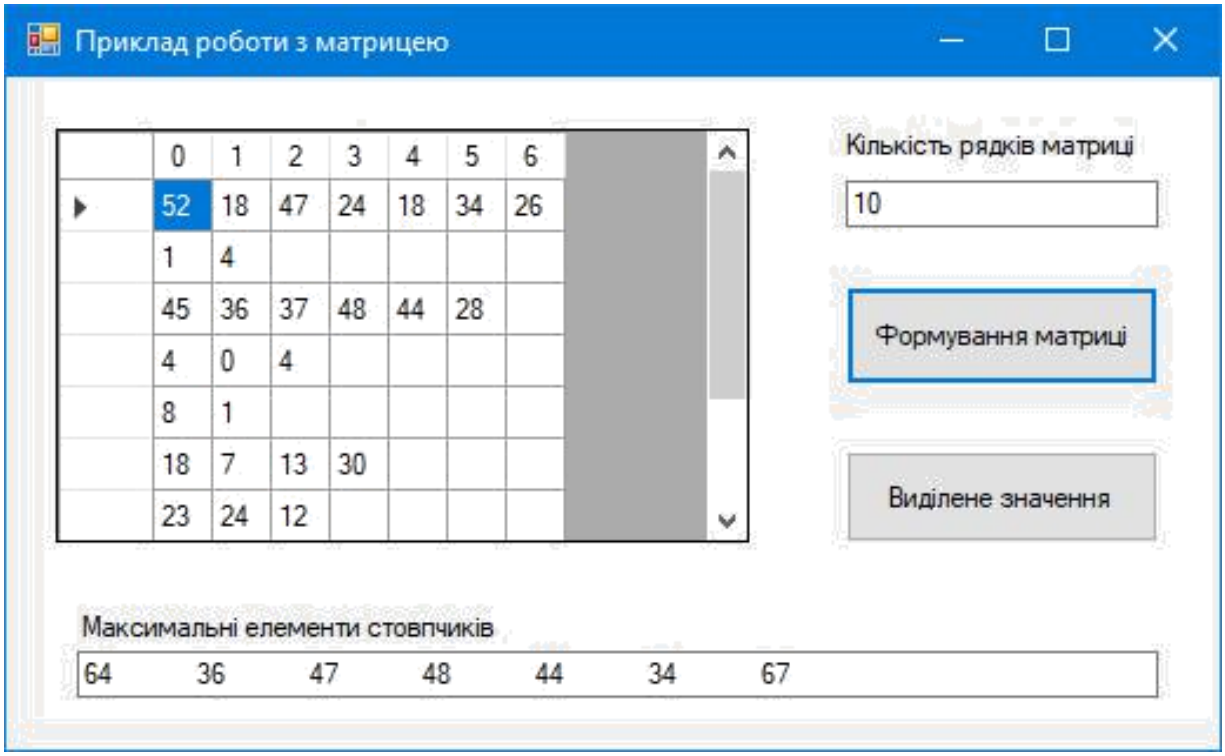


Рис. 10. Екранна форма прикладу 4

Порядок виконання роботи

1. Створити програму для розв’язання задачі формування та оброблення двовимірного динамічного масиву відповідно до варіанта. Додаток має містити головну форму, на якій знаходяться:
   - текстовий елемент керування з міткою для введення вхідних даних;
   - елементи керування *DataGridView* для виведення результатів;
   - кнопки формування вихідної і результуючої матриць.

2. Розробити окремі методи з параметрами:
   - формування динамічної квадратної матриці цілочислових даних;

– виведення на екран динамічної матриці в елемент керування DataGridView;

– видалення елементів (зі зміною розміру) динамічної матриці, заданих у варіанті.

Розмір початкової матриці N вводиться з клавіатури і фактично не обмежений (обмежений лише оперативною пам'яттю).

3. Опрацювати всі можливі виняткові ситуації.

Можливий вигляд форми показано на рис. 11.

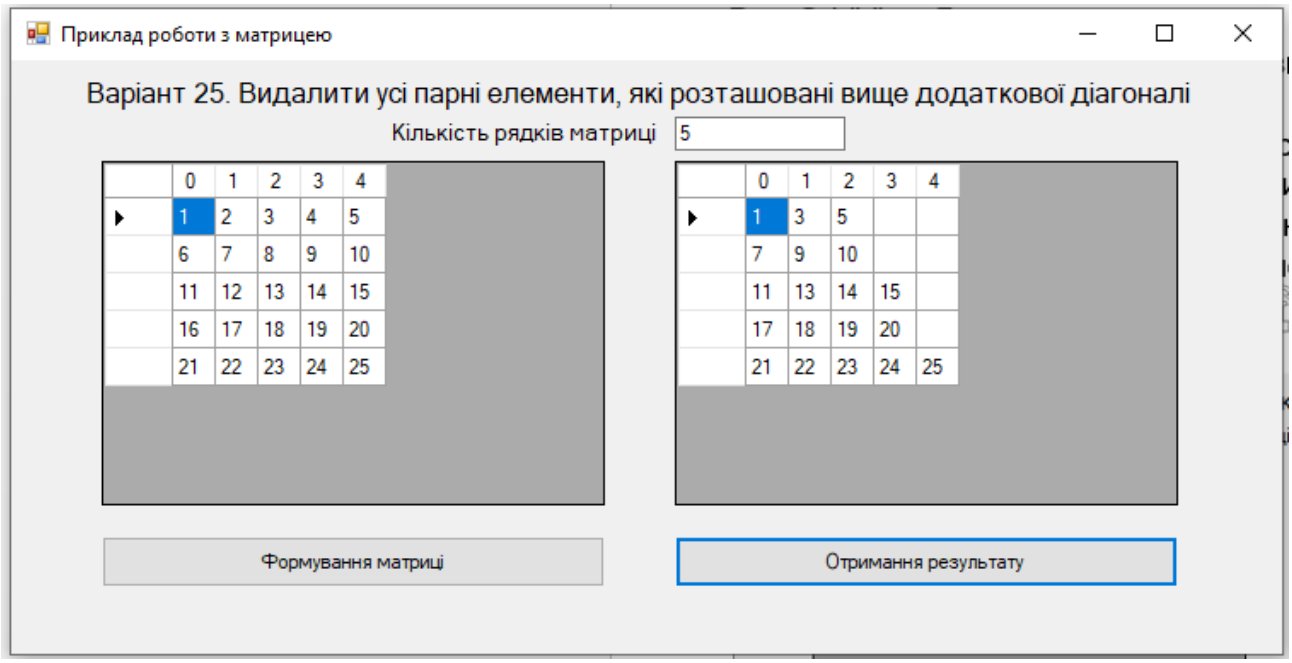


Рис. 11. Екранна форма лабораторної роботи № 4

Варіанти завдань до лабораторної роботи № 4

1. Видалити всі елементи, кратні 3:

1	2	3	:	n
n	n-1	n-2	:	1
1	2	3	:	n
n	n-1	n-2	:	1
...	...	...	...	...
n	n-1	n-2	:	1

2. Видалити всі елементи, що дорівнюють 0:

1	0	0	0	:	0	n
0	2	0	0	:	n-1	0
0	0	3	n-2	:	0	0
...	...	...		...		...
0	2	0	0	:	n-1	0
1	0	0	0	:	0	n

3. Видалити всі парні елементи:

0	0	:	0	0	0	1
0	0	:	0	0	2	0
0	0	:	0	3	0	0
...	...	...		...		...
0	n-1	:	0	0	0	0
n	0	:	0	0	0	0

4. Видалити всі непарні елементи:

1	2	3	:	n-2	n-1	n
2	1	2	:	n-3	n-2	n-1
3	2	1	:	n-4	n-3	n-2
...	...	...	...	...		...
n-1	n-2	n-3	:	2	1	2
n	n-1	n-2	:	3	2	1

5. Видалити всі нульові елементи в парних рядках:

n	0	0	:	0	0	0
0	n-1	0	:	0	0	0
0	0	n-2	:	0	0	0
...	...	...	...	...	...	...
0	0	0	...	0	2	0
0	0	0	:	0	0	1

6. Видалити всі елементи, що дорівнюють 0:

1	2	3	:	n-1	n
0	1	2	:	n-2	n-1
0	0	1	:	n-4	n-3
...	...	...	...	...	...
0	0	0	:	0	1

7. Видалити всі елементи, що дорівнюють 0 і розташовані вище головної діагоналі:

1	2	3	:	n-1	n
0	1	2	:	n-2	n-1
0	0	1	:	n-4	n-3
...	...	...	...	...	...
0	0	0	:	0	1

8. Видалити всі елементи, що дорівнюють 0, в непарних стовпчиках:

1·2	0	0	:	0	0
0	2·3	0	:	0	0
0	0	3·4	:	0	0
...	...	...	...		...
0	0	0	:	(n-1)·n	0
0	0	0	:	0	n·(n+1)

9. Видалити всі елементи, що дорівнюють 1 і розташовані нижче додаткової діагоналі:

1	1	1	:	1	1	1
1	0	0	:	0	0	1
1	0	0	:	0	0	1
...	...		...	...		...
1	0	0	:	0	0	1
1	0	0	:	0	0	0
1	1	1	:	1	1	1

10. Видалити всі елементи зліва від непарних значень:

0	0	0	:	0	0
0	1	0	:	0	0
0	0	2	:	0	0
...	...	...	...	...	...
0	0	0	:	0	n-1

11. Видалити всі елементи, що дорівнюють 0:

1	1	1	:	1	1	1
2	2	2	:	2	2	0
3	3	3	:	3	0	0
...	...		...	...		...
n-1	n-1	0	:	0	0	0
n	0	0	:	0	0	0

12. Видалити всі елементи, які більші за N / 2:

n	0	0	:	0	0	0
n-1	n	0	:	0	0	0
n-2	n-1	n	:	0	0	0
...	...	...	...	...	...	...
2	3	4	...	n-1	n	0
1	2	3	:	n-2	n-1	n

13. Видалити всі елементи, що дорівнюють 1 і розташовані вище головної діагоналі:

1	1	1	:	1	1	1
0	1	1	:	1	1	0
0	0	1	:	1	0	0
...	...		...	...		...
0	0	1	:	1	0	0
0	1	1	:	1	1	0
1	1	1	:	1	1	1

14. Видалити всі елементи, які менші за N / 2:

1	2	3	:	n-2	n-1	n
2	3	4	:	n-1	n	0
3	4	5	:	n	0	0
...	...	...	...	...	...	...
n-1	n	0	:	0	0	0
n	0	0	:	0	0	0

15. Видалити всі елементи, що дорівнюють 0 в парних рядках, і 1 - в непарних:

1	0	0	:	0	0	1
1	1	0	:	0	1	1
1	1	1	:	1	1	1
...	...		...	...	...	...
1	1	0	:	0	1	1
0	1	1	:	1	1	0
1	0	0	:	0	0	1

16. Видалити всі парні елементи:

1	0	0	:	0	0	n
0	2	0	:	0	n-1	0
0	0	3	:	n-2	0	0
...	...		...	...	...	...
0	2	0	:	0	n-1	0
1	0	0	:	0	0	n

17. Видалити всі елементи, кратні 2:

1	0	0	:	0	0	0
1	2	0	:	0	0	0
1	2	3	:	n-2	0	0
...	...		...	...	...	...
1	2	3	:	n-2	n-1	0
1	2	3	:	n-2	n-1	n

18. Видалити всі елементи, кратні 3:

0	0	0	:	0	0	1
0	0	0	:	0	2	1
0	0	0	:	3	2	1
...	...		...	...	...	...
0	0	n-2	:	3	2	1
n	n-1	n-2	:	3	2	1

19. Видалити всі ненульові елементи, які стоять справа від парних значень:

1	2	3	:	n-2	n-1	n
0	2	3	:	n-2	n-1	0
0	0	3	:	n-2	0	0
...	...	...	...	...	...	...
0	2	3	:	n-2	n-1	0
1	2	3	:	n-2	n-1	n

20. Видалити всі елементи, що дорівнюють 0 з непарною сумою індексів:

1	2	3	:	n-2	n-1	n
0	2	3	:	n-2	n-1	n
0	0	3	:	n-2	n-1	n
...	...	...	...	...	...	...
0	0	0	:	0	n-1	n
0	0	0	:	0	0	n

21. Видалити всі елементи, що дорівнюють 1 з парною сумою індексів:

1	1	1	:	1	1	1
1	2	1	:	1	1	1
1	2	3	:	1	1	1
...	...		...	...	...	...
1	2	3	:	n-2	n-1	1
1	2	3	:	n-2	n-1	n

22. Видалити k нульових елементів у k рядку:

0	0	0	0	:	0	n-1
0	0	0	0	:	n-2	0
0	0	0	n-3	:	0	0
...	...	...		...		...
0	1	0	0	:	0	0
0	0	0	0	:	0	0

23. Видалити всі нульові елементи, розташовані нижче головної діагоналі, і всі одиничні, що знаходяться нижче неї:

1	1	1	:	1	1	1
0	0	0	:	0	0	0
1	1	1	:	1	1	1
...	...	...		...		...
0	0	0	:	0	0	0
1	1	1	:	1	1	1

24. Видалити всі елементи, які стоять нижче непарного значення на головній діагоналі:

1	2	3	:	n-1	n
1	2^2	3^2	:	$(n-1)^2$	n^2
1	2^3	3^3	:	$(n-1)^3$	n^3
...	...	...	...	...	...
1	2^n	3^n	:	$(n-1)^n$	n^n

25. Видалити всі елементи, розташовані вище додаткової діагоналі:

1	2	3	:	n
n+1	n+2	n+3	:	2n
2n+1	2n+2	2n+3	:	3n
3n+1	3n+2	3n+3	:	4n
...	...	...	...	...
(n-1)(n+1)	(n-1)(n+2)	(n-1)(n+3)	:	n^2

Контрольні запитання

1. Опишіть особливості використання компонента `DataGridView` (Таблиця) для виведення матриці.
2. Назвіть основні властивості таблиці рядків.
3. Назвіть види двовимірних масивів у C#.
4. Опишіть особливості роботи з двовимірними масивами в C#.

Лабораторна робота № 5

ОБРОБЛЕННЯ РЯДКІВ. СТВОРЕННЯ МЕНЮ. ОБРОБЛЕННЯ ПОДІЙ КЛАВІАТУРИ

Мета роботи – вивчення алгоритмізації та програмування задач оброблення символьних даних; використання методів оброблення символьних даних; введення/виведення символьних даних та їх оброблення як масиву символів; робота з елементом керування `MenuStrip`; оброблення подій клавіатури.

Теоретичні відомості

Рядки

У мові програмування C# рядки є об'єктами, тобто string – це тип посилання. Для створення об'єкта типу string використовують такі форми запису:

```
string ім’я__змін;
string ім’я__змін = «значення рядка»;
char[]ім’я_масиву = {символи};
string ім’я__змін = new string(ім’я_масиву);
```

Наприклад:

```
string str1; // Об’являємо рядкову змінну
string str2 = "рядок"; // Об’являємо й ініціалізуємо рядок
// рядок створюється як масив символів
char[] arrchar = {’e’, ’x’, ’a’, ’m’, ’p’, ’l’, ’e’};
string str3 = new string(arrchar);
```

Тип даних `string` зазвичай пов'язаний з типом даних `char`, об'єкт `string` можна створити з масиву символів або перетворити на масив символів, але тип даних `string` і масив `char` – це різні типи даних. У мові програмування C# кожен рядок має довжину, яка змінюватися не може. Крім того, не можуть змінюватися символи, складові рядків. Тому рядки в C# називають постійними (immutable). Щоб змінити вміст, необхідно створити новий рядок. Безліч методів класу `string` створює новий рядок на основі існуючого.

Після створення `string` об'єкт можна застосовувати скрізь, де дозволяється використання рядка символів, взятого в лапки.

З рядками можна виконувати такі операції:

- призначення рядкових значень рядковим змінним;
- порівняння двох рядків;
- з’єднання (конкатенація) двох рядків;

– доступ до властивостей рядка і виклик різних методів, визначених для типу `string`.

Основні методи та властивості класу `string` наведено в табл. 13.

Таблиця 13

Основні методи і властивості класу System.string

Метод/властивість	Опис
Length	Властивість, що визначає довжину рядку (кількість символів)
IndexOf *IndexOfAny* *LastIndexOf* *LastIndexOfAny*	Методи, що визначають індекси першого і останнього входжень заданого підрядка або будь-якого символу із заданого набору
Copy *CopyTo*	Методи призначено для копіювання рядка або підрядка в інший рядок або масив типу `Char`
Substring	Метод виділення підрядка, починаючи із заданої позиції
Split	Метод, що розділяє рядок на елементи, використовуючи задані роздільники. Результат поміщають у масив рядків
Concat	Статичний метод конкатенації рядків. Допускає з’єднання довільної кількості рядків
Join	Статичний метод з’єднання масиву рядків у єдиний рядок. Між елементами масиву додаються розділові знаки
Insert	Метод вставки підрядка в задану позицію
Replace	Метод заміни всіх входжень заданого підрядка або символу новим підрядком або символом
Remove	Метод видалення підрядка із заданої позиції
PadLeft *PadRight*	Методи вирівнювання рядку по лівому або правому краю шляхом вставки потрібної кількості пропусків чи заданих символів на початку або в кінці рядку
Trim *TrimEnd* *TrimStart*	Методи, що видаляють пропуски або задані символи на початку або в кінці рядку, або тільки з одного його кінця

Закінчення табл. 13

ToLower *ToUpper*	Методи перетворення символів рядку до нижнього або верхнього регістру

Події натискання клавіш

У C# визначено події, пов’язані з клавіатурою, які наведено в табл. 14.

Таблиця 14

Події, пов’язані з клавіатурою

Подія	Опис
OnKeyDown	Відбувається при натисканні користувачем будь-якої клавіши. Можна розпізнати натиснуті клавіши, включаючи функціональні, і клавіши миші, але не можна розпізнати символ натиснутої клавіши
OnKeyPress	Відбувається при натисканні користувачем клавіші символу. Можна розпізнати тільки натиснуту клавішу символу, розрізнити символ у верхньому або нижньому регістрі, а також розрізнити букви кирилиці та латині, але не можна розрізнити функціональні клавіші та клавіши миші
OnKeyUp	Відбувається при відпусканні користувачем будь-якої клавіши. Можна розпізнати натиснуту клавішу, включаючи функціональні, та клавіши миші, але не можна розпізнати символ клавіші, яку відпускають

Приклад 5. Створимо програму, у вікно редагування якої користувачу дозволяється вводити лише цілі числа без знака. Екранну форму показано на рис. 12. Наведемо фрагмент коду програми, що обробляє подію введення даних у полі редагування:

```
    private      void      textBox1_KeyPress(object      sender,
KeyPressEventArgs e)
            {
//перевіряємо символ натиснутої клавіши
                if (!(char.IsDigit(e.KeyChar))) {
                   MessageBox.Show("Вводіть тільки цифри!!!");
                    e.KeyChar = '\0';
                }}
```

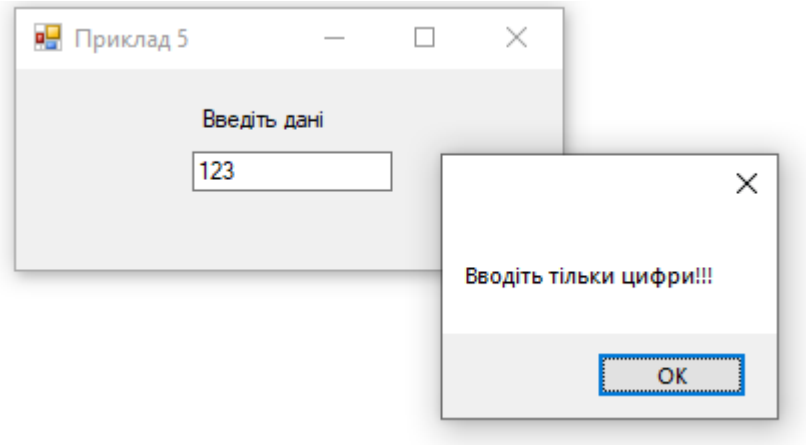


Рис. 12. Екранна форма прикладу 5

Елемент керування MenuStrip (меню)

Елемент керування *MenuStrip* (меню) надає системне меню для форми. Добавляється компонент за допомогою кнопки групи *Menus & ToolBars* (меню і панелі інструментів) панелі інструментів.

Порядок виконання роботи

Створити форму для введення даних, виведення результату. На форму помістити елемент керування *MenuStrip*, за допомогою якого організувати систему меню форми. Приклад форми показано на рис. 13.

Задати з клавіатури послідовність символів та реалізувати її оброблення, як зазначено у варіанті. Вхідні дані задати самостійно, враховуючи специфіку конкретного варіанта, написати оброблення події *KeyPress*.

Скласти програму оброблення набору символів (тексту) з використанням стандартних методів оброблення рядків.

Результат цього процесу та початкову послідовність вивести на екран.

У програмі мають бути передбачені методи введення вхідних даних з перевіркою на область допустимих значень, оброблення тексту.

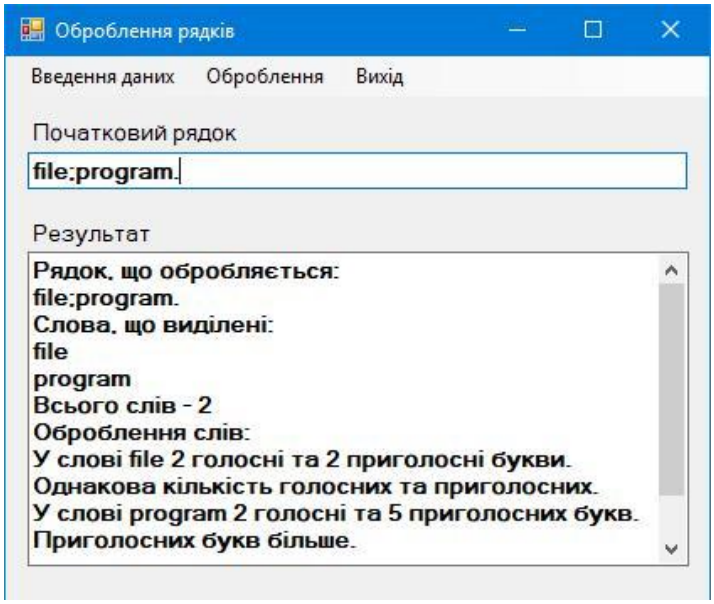


Рис. 13. Екранна форма лабораторної роботи № 5

Варіанти завдань до лабораторної роботи № 5

№ 1–5

Дано послідовність, яка містить слова, в кожному з яких від 1 до 5 прописних (великих) латинських букв, розділених не менше ніж одним пробілом, після останнього слова – крапка.

1. Указати слова, в яких немає символів, що повторюються.
2. Указати слова, в яких є символи, що повторюються.
3. Для кожного символу слова заданого тексту вказати, скільки разів він зустрічається в тексті. Повідомлення про цей символ має друкуватися не більше одного разу для одного слова.
4. Для кожного слова, заданого речення вказати частку голосних у відсотках (відношення кількості голосних до кількості букв у слові). Визначити слово, в якому частка голосних максимальна.
5. Знайти найдовше симетричне слово заданого речення, наприклад АККА.

№ 6–10

Дано послідовність, яка містить слова, в кожному з яких від 2 до 10 рядкових (маленьких) латинських букв, розділених комою, після останнього слова – крапка.

6. У заданому тексті знайти найкоротше і найдовше слово.

7. Із заданих слів речення вибрати і надрукувати тільки ті символи, що зустрічаються в ньому один раз (у тому порядку, в якому вони зустрічаються в слові). Наприклад, початкове речення: «a,volunteer,is,worth,twenty,pressed,men.», результат: «a,volunter,is,worth,tweny,presd,men.».
8. У кожному слові речення видалити середню букву, якщо це слово має непарну довжину.
9. У заданому тексті знайти слова «паліндроми» (паліндромом називається рядок, що однаково читається в обох напрямках (зліва направо та справа наліво). Наприклад, «level», «madam».
10. В усіх словах речення змінити місцями першу та останню літери.

№ 11—15

Дано послідовність, яка містить слова, в кожному з яких від 1 до 8 рядкових (маленьких) латинських букв, розділених крапкою з комою, після останнього слова – крапка.

11. Упорядкувати літери кожного слова речення за алфавітом.
12. Визначити, скільки разів задане слово входить у текст.
13. В усіх словах речення, що відрізняються від останнього слова, видалити всі попередні входження останньої букви. Наприклад, початкове речення: «pull;up;if;I;pull;up*»,* результат: *«*pul;up;if;I;pul;up*.».*
14. Надрукувати всі слова речення, які є частиною інших слів речення.
15. У довільному тексті видалити всі підряд символи, що повторюються, наприклад, із «*aaabbaa»* отримати «*aba»*.

№ 16–20

Дано послідовність, яка містить слова, в кожному з яких від 2 до 5 прописних (великих) латинських букв, розділених комою, після останнього слова – крапка.

16. Вивести всі слова заданого тексту в алфавітному порядку.
17. У словах виконати заміну регістру символів, тобто прописні букви замінити рядковими.
18. Усі букви, окрім першої, зробити прописними. Наприклад, із «*ONE,TWO,THREE.»* отримати *«One,Two,Three.».*
19. У заданому тексті видалити всі слова, що складаються із трьох і менше символів.
20. Перенести першу букву кожного слова в кінець.

№ 21–25

Дано послідовність, яка містить слова, в кожному з яких від 3 до 8 рядкових (маленьких) латинських букв, розділених крапкою з комою, після останнього слова – крапка.

21. Для кожного слова, заданого речення вказати частку голосних у відсотках (відношення кількості голосних до кількості букв у слові). Визначити слово, в якому частка голосних максимальна.
22. Визначити найкоротше слово і його довжину.
23. Визначити слова, в яких перша буква повторюється ще раз.
24. Визначити слова, в яких голосні букви чергуються із приголосними.
25. Перенести останню букву в початок слова.

№ 26–30

Дано послідовність, яка містить слова, в кожному з яких від 3 до 8 прописних (великих) латинських букв, розділених крапкою з комою, після останнього слова – крапка.

26. Для кожного слова визначити, яких букв в ньому більше: голосних чи приголосних.
27. Надрукувати слова, що починаються і закінчуються на одну й ту ж букву. Визначити їх кількість.
28. Визначити, скільки разів кожне слово зустрічається в тексті.
29. Вводяться два слова S_1 і S_2. Замінити в початковому тексті кожне входження слова S_1 на слово S_2.
30. Вводиться слово S. Визначити, скільки разів у початковому тексті зустрічається слово S.

Контрольні запитання

1. Опишіть особливості оброблення рядків.
2. Назвіть методи для оброблення рядків.
3. Опишіть особливості роботи з елементом керування *MenuStrip*.
4. Як виконується оброблення подій клавіатури?

Лабораторна робота № 6

ТЕКСТОВІ ФАЙЛИ (ПОТОКИ СИМВОЛІВ)

Мета роботи – вивчення введення/виведення з використанням файлів, особливості роботи з текстовими файлами.

Теоретичні відомості

Файли і потоки

Файл являє собою іменовану послідовність однотипних елементів, розміщених на зовнішньому пристрої. Кожен файл – це послідовність байтів. Файлу властиві ім'я та ряд інших характеристик, які розпізнає операційна система. У ці характеристики зазвичай входять такі дані, як час створення файла, час його останньої модифікації, час останнього звернення до нього, а також інформація про те, яким користувачам комп'ютера дозволено читати файл або записувати в нього.

Потік (stream) – абстрактне поняття, що належить до будь-якого перенесення даних від джерела до приймача.

Типи, необхідні для реалізації потоків в C#, а також усі методи містяться в двох просторах імен:

- *System* має клас *Console* для взаємодії з екраном і клавіатурою;
- *System.IO* має класи для взаємодії з файлами.

Виконувати обмін із зовнішніми пристроями можна на рівні:

- двійкового подання *(BinaryReader, BinaryWriter)*;
- байтів (FileStream);
- тексту *(StreamWriter, StreamReader)*.

Текстові потоки

У .Net використовується кодування Unicode, де кожен символ кодується двома байтами. Класи, що працюють з текстом, є оболонками класів, які застосовують байти, і автоматично перетворюють байти на символи, і навпаки. Доступ до таких файлів послідовний, коли черговий елемент можна почитати (записати) тільки після аналогічної операції з попереднім елементом.

Використання класів текстових потоків у програмі для читання передбачає такі операції:

1. Створення текстового потоку і зв'язування його з фізичним файлом:

```
StreamReader < ім’я потоку> = new StreamReader(<ім’я файлу>);
```

2. Введення даних:

```
while ((<рядок>=<ім’я потоку>.ReadLine())!=null)
     {<тіло циклу>}
```

3. Закриття потоку:

```
<ім’я потоку>.Close();
```

Використання класів текстових потоків у програмі для запису дозволяє такі операції:

1. Створення текстового потоку і зв'язування його з фізичним файлом:

```
StreamWriter <ім’я потоку> = new StreamWriter (<ім’я
файла>);
```

2. Введення даних:

```
<ім’я потоку>.WriteLine(<рядок>);
```

3. Закриття потоку:

```
<ім’я потоку>.Close();
```

Елементи керування для роботи з файлами

Панель інструментів у групі *Dialogs* (діалогові вікна) містить компоненти, призначені для відображення стандартних діалогових вікон «Відкрити файл» і «Зберегти у файл» операційної системи Windows.

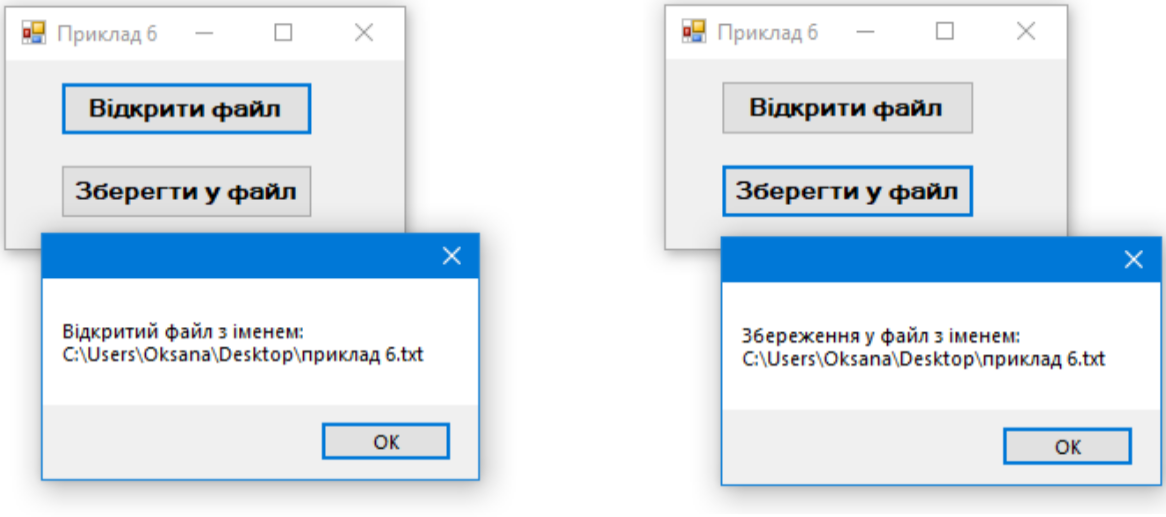
Компонент Windows Forms *OpenFileDialog* є стандартним діалоговим вікном. Він аналогічний діалоговому вікну «Відкриття файлу» операційної системи Windows, успадковується від класу *CommonDialog* і додається на форму за допомогою кнопки .

Компонент Windows Forms *SaveFileDialog* є стандартним діалоговим вікном. Він аналогічний стандартному діалоговому вікну «Зберегти у файл», що використовується в операційних системах Windows, успадковується від класу *CommonDialog* і додається на форму за допомогою кнопки .

Приклад 6. Створити програму, в якій при натисканні кнопок відображаються діалогові вікна «Відкрити файл» і «Зберегти у файл». Екранну форму показано на рис. 14. Наведемо фрагмент коду програми, що опрацьовує події натискання на кнопки:

```
private void button1_Click(object sender, EventArgs
e) {
    if (openFileDialog1.ShowDialog()==DialogResult.OK)
       MessageBox.Show("Відкритий файл з іменем:\r\n" +
openFileDialog1.FileName);
}
```

```
    private void button2_Click(object sender, EventArgs
    e) {
       if (saveFileDialog1.ShowDialog()==DialogResult.OK)
         MessageBox.Show("Збереження у файл з іменем:\r\n" +
saveFileDialog1.FileName);}
```



а б

Рис. 14. Екранні форми прикладу 6: а – відкриття файла;
б – збереження файла

Порядок виконання роботи

Створити форму, що має меню і компоненти для відображення стандартних діалогових вікон «Відкрити файл» і «Зберегти у файл». Приблизний вигляд форми показано на рис. 15. Взяти за основу завдання з лабораторної роботи № 5 та прочитати послідовність символів з текстового файла, реалізувати її оброблення, як зазначено у варіанті. Результат оброблення вивести в результуючий текстовий файл. Вхідні дані задати самостійно, враховуючи специфіку конкретного варіанта. Кількість речень у тексті може бути довільною. Обробити можливі винятки.

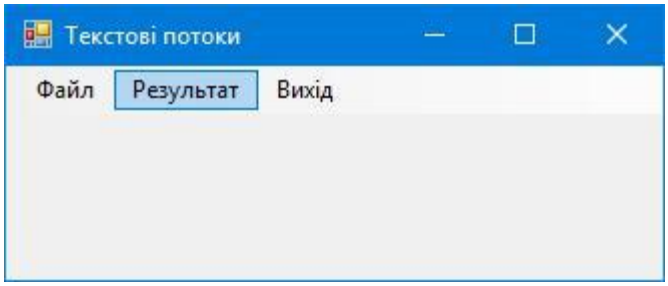


Рис. 15. Екранна форма лабораторної роботи № 6

Контрольні запитання

1. Опишіть особливості виконання операторів: циклів, умовних, введення/ виведення.
2. Назвіть методи оброблення рядків.
3. Опишіть особливості роботи з текстовими потоками.

Лабораторна робота № 7

ДВІЙКОВІ ФАЙЛИ (ПОТОКИ)

Мета роботи – вивчення введення/виведення з використанням файлів, особливості роботи з двійковими файлами.

Теоретичні відомості

Робота з бінарними файлами

Двійкові потоки зберігають дані в тому ж вигляді, в якому вони подані в оперативній пам'яті, тобто у внутрішній формі подання. Вони призначені для використання в програмах, а не для перегляду. Вихідний потік *BinaryWriter* підтримує довільний доступ на відміну від текстових потоків, тобто є можливість виконувати запис у довільну позицію двійкового файла. Двійковий файл відкривається на основі базового потоку *FileStream*. Вхідний двійковий потік містить перевантажені методи читання для всіх простих вбудованих типів даних.

Використання класів бінарних потоків у програмі для читання передбачає такі операції:

1. Створення базового файлового потоку і зв'язування його з фізичним файлом:

```
FileStream <ім’я базового потоку> = new FileStream (<ім’я
файла>);
```

2. Створення вхідного двійкового потоку на базі створеного файлового потоку:

```
BinaryReader <ім’я потоку для читання> = new BinaryReader
(<ім’я базового потоку>);
```

3. Введення даних здійснюється в нескінченному циклі, вихід з якого відбувається за винятком EndOfStreamException (кінець потоку). Для читання з потоку даних певного типу використовуються методи `ReadXXX` (наприклад, *ReadInt32, ReadByte, ReadBoolean).*

Загальний вигляд циклу:

```
    try
       {while (true)// нескінченний цикл, вихід з циклу за
винятком
         <тип елемента> el = <ім’я потоку для
читання>.ReadXXX(); // читання
    <оброблення елемента файла>}
```

```
    catch (EndOfStreamException ex)
{
        }
```

4. Закриття потоку:

```
<ім’я потоку>.Close();
```

Використання класів бінарних потоків у програмі для запису допускає такі операції:

1. Створення базового файлового потоку і зв'язування його з фізичним файлом:

```
FileStream <ім’я базового потоку> = new FileStream (<ім’я файла>);
```

2. Створення вхідного двійкового потоку на базі створеного файлового потоку:

```
BinaryWriter <ім’я потоку для запису> = new BinaryWriter (<ім’я базового потоку>);
```

3. Виведення даних:

```
<ім’я потоку для запису >.Write(<елемент>);
```

4. Закриття потоку:

```
<ім’я потоку>.Close();
```

Вихідний потік *BinaryWriter* підтримує довільний доступ, для цього є можливість переміщати покажчик файла в довільну позицію двійкового файла. Для відкритого вихідного двійкового потоку це можна зробити за допомогою такого методу:

```
<ім’я потоку для запису >.Seek(<кількість байтів>, <позиція у файлі>);
```

Параметр `<позиція у файлі>` може набувати таких значень:

`SeekOrigin.Begin` – початок файла;
`SeekOrigin.Current` – поточна позиція;
`SeekOrigin.End` – кінець файла.

Наприклад*:*

– у двійковому потоці Fout переміщуємо покажчик на чотири байти перед поточною позицією:

```
Fout.Seek(-4, SeekOrigin.Current);
```

– у двійковому потоці `Fout` ставимо на початок файла:

```
fileStream.Seek(0, SeekOrigin.Begin);
```

Клас FileStream

Клас *FileStream* розширює базовий клас *Stream* (потік) для роботи з файлом, який підтримує синхронні й асинхронні операції читання і запису. Створення потоку можливе за допомогою конструктора:

```
FileStream(string path,FileMode mode,
FileAccess access,FileShare share,int bufferSize,bool useAsync)
```

Параметри позначають таке:

`path` – абсолютне або відносне ім'я файла;

`mode` – константа `FileMode`, що визначає спосіб відкриття або створення файла і може набути значення (табл. 15);

`access` – константа `FileAccess`, що визначає доступ до файла і може приймати значення (табл. 16);

`share` – константа `FileShare` визначає спільний доступ до файла;

`bufferSize` – додатне значення `Int32` більше 0, що визначає розмір буфера. Для значень `bufferSize` від 0 до 8 фактичний розмір буфера встановлюют таким, щоб він дорівнював 8 байтам;

`useAsync` – вказує, чи використовувати асинхронне або синхронне введення/виведення. Однак основна операційна система може не підтримувати асинхронне введення/виведення, тому, коли задається значення true, дескриптор може бути відкритий синхронно залежно від платформи.

Таблиця 15

Значення параметра `mode` константи `FileMode`

Значення	Опис
CreateNew	Створення нового файла, якщо файл уже існує, то створюється виняток *`IOException`*
Create	Створення нового файла, якщо файл уже існує, то він буде перезаписаний. Значення еквівалентне такому запиту: якщо файл не існує, то використовувати *`CreateNew`*; в іншому разі – *`Truncate`*
Open	Відкриття існуючого файла. Можливість відкрити цей файл залежить від значення, що задається *`FileAccess`*. Якщо файл не існує, то створюється виняток *`FileNotFoundException`*

Закінчення табл. 15

Значення	Опис
OpenOrCreate	Відкриття файла, якщо він існує, інакше, повинен бути створений новий файл
Truncate	Відкриття існуючого файла та його усічення так, щоб його розмір дорівнював нулю
Append	Відкриття файла, якщо він існує, і виконання пошуку до кінця файла або ж створення нового файла

Таблиця 16

Значення параметра `access` константи `FileAccess`

Значення	Опис
Read	Дані можна прочитати із файла
Write	Дані можна записати у файл
ReadWrite	Дані можна записати у файл і прочитати із файла

Порядок виконання роботи

Розробити програму з віконним інтерфейсом для створення початкового двійкового файла чисел (тип чисел зазначено у варіанті) з довільною кількістю елементів.

Передбачити можливість задавати кількість елементів та їх значення двома способами:

- з клавіатури;
- за допомогою генератора випадкових чисел.

Вхідні дані, структуру файла та їх оброблення реалізувати згідно з заданим варіантом. При реалізації варіанта масиви використовувати ЗАБОРОНЯЄТЬСЯ. Передбачити можливість виведення результату на екран і в текстовий файл.

У програмі мають бути передбачені такі підпрограми:

- введення елементів із файла;
- оброблення даних;
- виведення результатів.

Передбачити оброблення виняткових ситуацій.

Можливий вигляд форми показано на рис. 16.

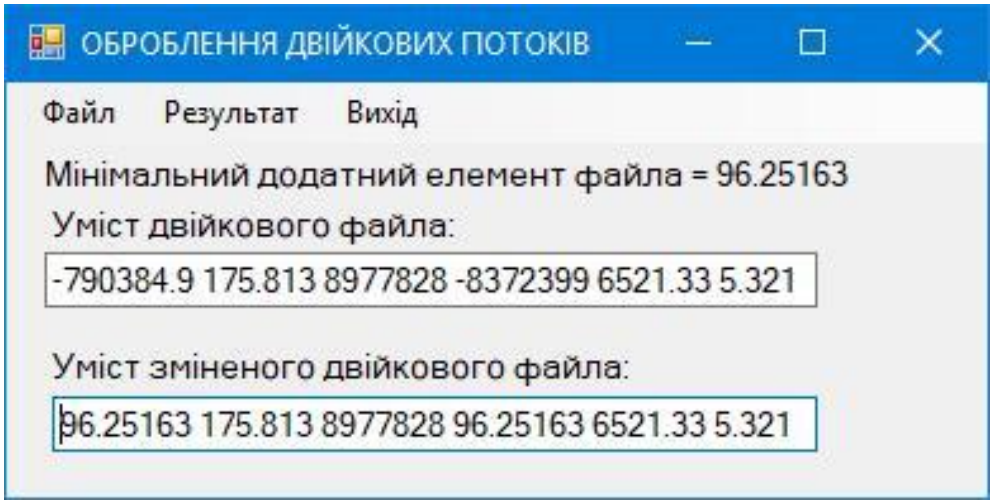


Рис. 16. Екранна форма лабораторної роботи № 7

Варіанти завдань до лабораторної роботи № 7

№ 1–5

Прочитати з файла *дійсні чотирибайтні* елементи, підрахувати загальну кількість елементів файла, а також замінити у файлі:

1) усі нульові елементи на середнє арифметичне всіх елементів файла;
2) усі додатні елементи на суму від'ємних елементів файла;
3) усі від'ємні елементи на кількість елементів у найбільш довгій зростаючій послідовності файла;
4) усі найменші елементи на найбільший елемент з файла;
5) усі максимальні елементи на добуток усіх додатних чисел.

№ 6–10

Прочитати з файла *цілі чотирибайтні* елементи, підрахувати загальну кількість елементів файла, а також замінити у файлі:

6) усі парні елементи на суму квадратів компонентів файла;
7) усі непарні елементи на модуль суми компонентів файла;
8) усі максимальні за модулем елементи на кількість парних елементів файла;
9) усі парні елементи на добуток непарних елементів;
10) усі мінімальні за модулем елементи на різницю між першою і останньою компонентами.

№ 11–15

Прочитати з файла *дійсні восьмибайтні* елементи, підрахувати загальну кількість елементів файла, а також замінити у файлі:

11) усі від'ємні елементи на середнє арифметичне всіх додатних елементів файла;
12) усі елементи, модуль яких менше одиниці, на добуток від'ємних елементів файла;
13) усі додатні елементи на кількість елементів у найдовшій спадній послідовності файла;
14) усі елементи, які менше -1, на найменший елемент з файла;
15) усі від'ємні елементи на суму всіх додатних чисел.

№ 16–20

Прочитати з файла *цілі двобайтні* елементи, підрахувати загальну кількість елементів файла, а також замінити у файлі:

16) усі нульові елементи на суму елементів файла, що мають парні номери;
17) усі елементи з парними номерами на модуль суми елементів файла, що мають непарні номери;
18) усі елементи, кратні трьом, на суму парних елементів файла;
19) усі мінімальні елементи на кількість непарних елементів;
20) усі максимальні елементи на номер найменшого елемента файла.

№ 21–25

Прочитати з файла *цілі восьмибайтні* елементи, підрахувати загальну кількість елементів файла, а також замінити у файлі:

21) усі мінімальні елементи на номер найбільшого елемента файла;
22) усі від'ємні елементи на найбільший елемент файла з парним номером;
23) усі від'ємні елементи на найменший елемент файла з непарним номером;
24) усі елементи, які менше середнього арифметичного, на кількість від'ємних елементів;
25) усі елементи, які більші середнього арифметичного, на найбільший від'ємний елемент.

№ 26–30

Прочитати з файла *цілі однобайтні* елементи, підрахувати загальну кількість елементів файла, а також замінити у файлі:

26) усі від'ємні елементи на найменший додатний елемент;
27) усі елементи, кратні п'яти, на найменший парний елемент файла;
28) усі додатні елементи на найбільший непарний елемент файла;

29) усі нульові елементи на суму номерів найбільшого і найменшого елементів файла;
30) усі парні елементи, більші десяти, на добуток першого додатного і останнього від'ємного елемента.

Контрольні запитання

1. Опишіть особливості виконання операторів: циклів, умовних, введення/ виведення.
2. Назвіть стандартні функції роботи з файлами.
3. Опишіть особливості роботи з двійковими потоками введення/ виведення.

Лабораторна робота № 8

РОБОТА З ОБ’ЄКТАМИ. ІНКАПСУЛЯЦІЯ. УСПАДКУВАННЯ. ПОЛІМОРФІЗМ

Мета роботи – набуття навичок створення власних класів.

Теоретичні відомості

Поняття класу та об'єкта

Клас – це складна структура, що містить, крім опису даних, опис методів, які можуть бути виконані над представником класу – об'єктом (екземпляром класу).

Клас оголошується за допомогою ключового слова class:

```
[<модифікатори>] class <ідентифікатор> [: <базовий клас>]
         {<тіло класу>}
```

Приклад оголошення простого класу:

```
public class point
{
     double _x;
     public void Show()
     {
          Console.WriteLine("Координати
      точки x="+_x.ToString());
     }
}
```

Дані класу називають *полями,* а функції – *методами*. В наведеному прикладі `point` – ім'я класу, `_x` – ім'я поля, `Show` – ім'я методу.

Об'єкти створюються явним або неявним чином, тобто або програмістом, або системою. Програміст створює екземпляр класу за допомогою операції `new`, тобто клас належить до посилальних типів. Загальний вигляд:

```
<ім’я класу> <ідентифікатор> = new <ім’я класу>([параметри]);
```

Наприклад*:*

```
point x = new point ();
```

Конструктор ініціалізує об'єкт при його створенні. Він має таке ж ім'я, як і сам клас, а синтаксично є подібним до методу. Всі класи мають

конструктори незалежно від того, визначите Ви їх чи ні, оскільки C# автоматично дає *конструктор за замовчуванням*, який ініціалізує всі змінні члени, які мають тип значень, нулями, а змінні члени типу посилання – null-значеннями. При визначенні власного конструктора конструктор за замовчуванням більше не використовується.

Формат запису конструктора такий:

```
<модифікатор доступу> ім'я_класу(<параметри>) { тіло конструктора }
```

Метод

Методи класу синтаксично є звичайними функціями мови. Їх опис задовольняє звичайним правилам оголошення функцій. Змістовно методи визначають абстракцію даних, яку реалізує клас. Вони описують операції, що можна виконати над об'єктами класу. Два об'єкти одного класу мають один і той же набір методів.

Ім'я методу спільно з кількістю, типами і специфікаторами його параметрів являє собою *сигнатуру методу,* тобто ту, за якою один метод відрізняють від інших. У класі не має бути методів з однаковими сигнатурами.

Мова C# дозволяє типу перевантажувати його методи, тобто коли клас має декілька членів з однаковими іменами, що відрізняються сигнатурою, відповідний член називають *перевантаженим*.

В операторах методу доступ до полів об'єкта здійснюється без зазначення імені об'єкта.

Інкапсуляція та властивості об'єкта

Під *інкапсуляцією* розуміють приховування полів об'єкта з метою забезпечення доступу до них тільки за допомогою методів класу.

Принцип інкапсуляції припускає, що до внутрішніх даних об'єкта не можна звернутися безпосередньо через екземпляр цього об'єкта. Замість цього для отримання інформації про внутрішній стан об'єкта та внесення змін необхідно використовувати спеціальні методи.

В C# інкапсуляція реалізується двома способами:

1) за допомогою *ключових слів* `public`, `private` і `protected` (на рівні синтаксису);

2) за допомогою *властивостей об'єкта*. Властивість містить поле і методи доступу до цього поля. Вона складається з імені і пари аксесорів (`get` і `set`). Аксесор використовують для читання вмісту змінної і запису в неї нового значення. Формат запису властивостей:

```
<тип> <ім'я>
{
get
{
     // код аксесора читання поля
}
[set
{
     // код аксесора запису поля
}]
}
```

Тут <тип> – це тип властивості, а <ім'я> – його ім'я. Після визначення властивості будь-яке використання імені типа означає виклик відповідного аксесора. Аксесор set автоматично приймає параметр з ім'ям value (ключове слово), який містить значення, що присвоюється властивості.

Наприклад*:*

```
public class point
{
     double _x;
     public double X //властивість
     {
          get
          {
               return _x;
          }
          set
          {
               _x = value;
          }
     }
}
```

У програмі для установлення значення властивості записується оператор присвоювання значення властивості.

Наприклад*:*

```
point T1 = new point(); //створення об'єкта
T1.X = -5; // присвоювання властивості значення
```

Спадкування

Механізм отримання нових класів за допомогою додавання полів, властивостей і методів до вже існуючих класів називається *спадкуванням*. При цьому новий породжений клас (нащадок) успадковує властивості й методи свого базового, батьківського класу. В оголошенні класу-нащадка

зазначається клас батька. Загальна форма оголошення класу, який успадковує базовий клас:

```
class ім'я__похідного_класу :
ім'я_базового_класу {
// тіло класу
}
```

Наприклад, клас `pointXY` (точка на площині) може бути породжений від розглянутого вище класу `point` шляхом додавання поля `_y` (ордината) і властивості `Y`. Оголошення класу `pointXY` у цьому випадку має такий вигляд:

```
public class pointXY: point
{
    double _y;
    public double Y
    {
        get
        {
            return _y;
        }
        set
        {
            _y = value;
        }
    }
```

Поліморфізм і віртуальні методи

Поліморфізм – це можливість застосовувати однакові імена для методів, що належать до різних класів. Концепція поліморфізму забезпечує в разі застосування методу до об'єкта використання саме того методу, який відповідає класу об'єкта. Для реалізації поліморфізму застосовують ключові слова C#: `virtual` і `override`. Якщо базовий клас визначає метод, який має бути заміщений у похідному класі, то цей метод має бути оголошений як віртуальний `(virtual)`.

Наприклад, визначено класи, один з яких є базовим для двох інших:

```
//клас, що визначає точку на лінії
public class point
{…
    public point(double x)//конструктор базового класу
    {
        _x = x;
    }
    public virtual double length()
```

```
        {
            return (Math.Abs(X));
        }
    }
    //клас, що визначає точку на площині
    public class pointXY: point
    {…
    //конструктор похідного класу

        public pointXY(double x, double y)
            : base(x)
        {
            _y = y;
        }
        public override double length()
        {
            return Math.Sqrt(X * X + Y * Y);
        }
    }
```

У кожному з цих класів визначено метод `length`. У базовому класі за допомогою директиви `virtual` метод `length` оголошено таким, що він дає можливість віртуальному і дочірньому класам замінити *віртуальний* метод своїм власним.

У кожному дочірньому класі визначено свій метод `length`, позначений директивою `override`, який заміщає відповідний метод батьківського класу.

Модифікатори, що використовують при оголошенні класу

Модифікатори визначають доступність класу або його елементів для інших елементів програми:

- *public* (відкриті);
- *protected* (захищені);
- *internal* (внутрішні);
- *published* (опубліковані);
- *private* (закриті);
- *automated* (автоматизовані).

Public не накладає обмежень на область видимості полів, методів і властивостей, що перелічуються в ній, їх можна викликати в будь-якому іншому модулі програми.

Published також не обмежує область видимості, проте в ній перелічуються властивості, які мають бути доступними не тільки на етапі виконання, але й на етапі конструювання програми (у вікні інспектора об'єктів).

Private звужує область видимості до мінімуму: елементи опису, зазначені в цій секції, доступні тільки всередині методів даного класу і підпрограм, що знаходяться в тому ж модулі, де описано клас. Елемент, оголошений в ***private***, не доступний навіть найближчим нащадкам класу, якщо вони розміщуються в інших модулях.

Protected надає доступ тільки методам самого класу, а також будь-яким його нащадкам незалежно від того, знаходяться вони в тому ж модулі чи ні.

Automated використовується для оголошення властивостей і методів, які будуть додані до так званого інтерфейсу OLE-об'єктів автоматизації, область видимості членів цієї секції не обмежена.

Порядок виконання роботи

1. Розробити програму з віконним інтерфейсом (з використанням успадкування, інкапсуляції та поліморфізму), яка дозволяє:
   - створювати динамічний масив об’єктів класу відповідно до варіанта;
   - проглядати список;
   - демонструвати застосування методів до об’єктів.
2. Розмістити опис властивостей і методів класу і підкласів в окремому модулі. Клас повинен мати загальні властивості і методи, а підкласи – свої властивості.
3. Створити віртуальні методи для формування рядка з описом об’єкта і реалізації обчислень.
4. Передбачити оброблення найбільш типових виняткових ситуацій.

Можливий вигляд головної форми показано на рис. 17.

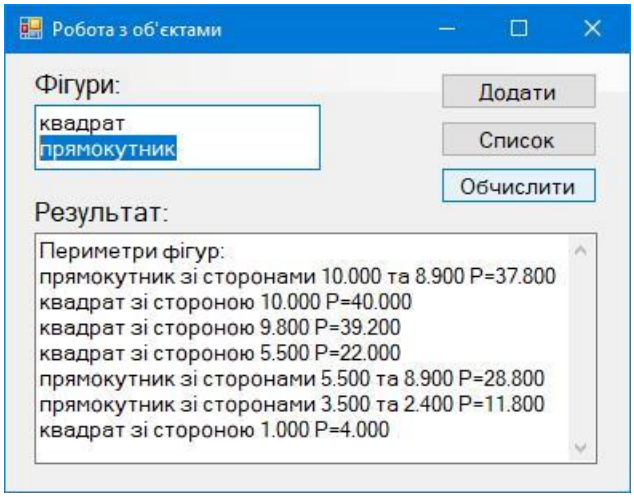


Рис. 17. Вигляд форми лабораторної роботи № 8

Варіанти завдань до лабораторної роботи № 8

1. Клас «Плоскі фігури», що має властивості (висота, основа) і методи формування рядка з описом об’єкта і обчислення площі. Описати класи-

нащадки – трикутник і паралелограм. Для обчислення площ скористатися формулами

$$S_{\text{трикутника}} = \frac{1}{2}ah, \qquad S_{\text{паралелограма}} = ah.$$

2. Клас «Плоскі фігури», що має властивості (сторона 1, сторона 2) і методи формування рядка з описом об'єкта і обчислення периметра. Описати класи-нащадки: трикутник з властивістю (сторона 3) і паралелограм. Для обчислення периметрів скористатися формулами

$$P_{\text{трикутника}} = a + b + c, \qquad P_{\text{паралелограма}} = 2(a + b).$$

3. Клас «Паралелограми», що має властивість (сторона 1) і методи формування рядка з описом об'єкта і обчислення периметра. Описати класи-нащадки: прямокутник з властивістю (сторона 2) і квадрат. Для обчислення периметрів скористатися формулами

$$P_{\text{прямокутника}} = 2(a + b), \qquad P_{\text{квадрата}} = 4a.$$

4. Клас «Паралелограми», що має властивість (сторона 1) і методи формування рядка з описом об'єкта і обчислення площі. Описати класи-нащадки: прямокутник з властивістю (сторона 2) і квадрат. Для обчислення площ скористатися формулами

$$S_{\text{прямокутника}} = ab, \qquad S_{\text{квадрата}} = a^2.$$

5. Клас «Чотирикутники», що має властивості (сторона 1, висота) і методи формування рядка з описом об'єкта і обчислення площі. Описати класи-нащадки: трапеція з властивістю (сторона 2) і ромб. Для обчислення площ скористатися формулами

$$S_{\text{трапеції}} = \frac{1}{2}(a + b)h, \qquad S_{\text{ромба}} = ah.$$

6. Клас «Круглі тіла», що має властивість (радіус) і методи формування рядка з описом об'єкта і обчислення об'єму. Описати класи-нащадки: циліндр з властивістю (висота) і сфера. Для обчислення об'єму скористатися формулами

$$V_{\text{циліндра}} = \pi R^2 h, \qquad V_{\text{сфери}} = \frac{4}{3}\pi R^3.$$

7. Клас «Круглі тіла», що має властивість (радіус) і методи формування рядка з описом об’єкта і обчислення площі повної поверхні. Описати класи-нащадки: циліндр з властивістю (висота) і сфера. Для обчислення площі повної поверхні скористатися формулами

$$S_{\text{циліндра}} = 2\pi R(R + h), \qquad S_{\text{сфери}} = 4\pi R^2.$$

8. Клас «Круглі тіла», що має властивість (радіус) і методи формування рядка з описом об’єкта і обчислення площі повної поверхні. Описати класи-нащадки: конус з властивістю (висота) і сфера. Для обчислення площі повної поверхні скористатися формулами

$$S_{\text{конуса}} = \pi R(R + \sqrt{h^2 + R^2}), \qquad S_{\text{сфери}} = 4\pi R^2.$$

9. Клас «Круглі тіла», що має властивість (радіус) і методи формування рядка з описом об’єкта і обчислення об’єму. Описати класи-нащадки: конус з властивістю (висота) і сфера. Для обчислення об’єму скористатися формулами

$$V_{\text{конуса}} = \frac{1}{3}\pi R^2 h, \qquad V_{\text{сфери}} = \frac{4}{3}4\pi R^3.$$

10. Клас «Круглі тіла», що має властивості (радіус, висота) і методи формування рядка з описом об’єкта і обчислення об’єму. Описати класи-нащадки: конус і циліндр. Для обчислення об’єму скористатися формулами

$$V_{\text{конуса}} = \frac{1}{3}\pi R^2 h, \qquad V_{\text{цилінда}} = \pi R^2 h.$$

11. Клас «Круглі тіла», що має властивості (радіус, висота) і методи формування рядка з описом об’єкта і обчислення площі повної поверхні. Описати класи-нащадки: конус і циліндр. Для обчислення площі повної поверхні скористатися формулами

$$S_{\text{конуса}} = \pi R(R + \sqrt{h^2 + R^2}), \qquad S_{\text{циліндра}} = 2\pi R(R + h).$$

12. Клас «Паралелепіпед», що має властивість (сторона 1) і методи формування рядка з описом об’єкта і обчислення площі повної поверхні. Описати класи-нащадки: прямокутний паралелепіпед з властивостями (сторона 2, сторона 3) і куб. Для обчислення площі повної поверхні скористатися формулами

$$S_{\text{прям. парал}} = 2(ab + bc + ca), \qquad S_{\text{куба}} = 6a^2.$$

13. Клас «Паралелепіпед», що має властивість (сторона 1) і методи формування рядка з описом об’єкта і обчислення об’єму. Описати класи-нащадки: прямокутний паралелепіпед з властивостями (сторона 2, сторона 3) і куб. Для обчислення об’єму скористатися формулами

$$V_{\text{прям. парал}} = abc, \qquad V_{\text{куба}} = a^3.$$

14. Клас «Плоскі фігури», що має властивість (сторона) і методи формування рядка з описом об’єкта і обчислення площі. Описати класи-нащадки: трикутник з властивістю (висота) і квадрат. Для обчислення площ скористатися формулами

$$S_{\text{трикутника}} = \frac{1}{2}ah, \qquad S_{\text{квадрата}} = a^2.$$

15. Клас «Плоскі фігури», що має властивість (сторона 1) і методи формування рядка з описом об’єкта і обчислення периметра. Описати класи-нащадки: трикутник з властивостями (сторона 2 і сторона 3) і квадрат. Для обчислення периметрів скористатися формулами

$$P_{\text{трикутника}} = a + b + c, \qquad P_{\text{квадрата}} = 4a.$$

16. Клас «Плоскі фігури», що має властивість (сторона 1) і методи формування рядка з описом об’єкта і обчислення площі. Описати класи-нащадки: прямокутник з властивістю (сторона 2) і паралелограм з властивістю (висота). Для обчислення площ скористатися формулами

$$S_{\text{паралелограма}} = ah, \qquad S_{\text{прямокутника}} = ab.$$

17. Клас «Плоскі фігури», що має властивість (сторона 1) і методи формування рядка з описом об’єкта і обчислення площі. Описати класи-нащадки: трапеція з властивостями (сторона 2 і висота) і квадрат. Для обчислення площ скористатися формулами

$$S_{\text{трапеції}} = \frac{1}{2}(a + b)h, \qquad S_{\text{квадрата}} = a^2.$$

18. Клас «Плоскі фігури», що має властивість (сторона 1) і методи формування рядка з описом об’єкта і обчислення площі. Описати класи-нащадки: трикутник з властивістю (висота) і прямокутник з властивістю (сторона 2). Для обчислення площ скористатися формулами

$$S_{\text{трикутника}} = \frac{1}{2}ah, \qquad S_{\text{прямокутника}} = ab.$$

19. Клас «Плоскі фігури», що має властивості (сторона 1, сторона 2) і методи формування рядка з описом об’єкта і обчислення периметра. Описати класи-нащадки: трикутник з властивістю (сторона 3) і прямокутник. Для обчислення периметрів скористатися формулами.

$$P_{\text{трикутника}} = a + b + c, \qquad P_{\text{прямокутника}} = 2(a + b).$$

20. Клас «Плоскі фігури», що має властивості (сторона 1, висота) і методи формування рядка з описом об’єкта і обчислення площі. Описати класи-нащадки: трапеція з властивістю (сторона 2) і паралелограм. Для обчислення площ скористатися формулами

$$S_{\text{трапеції}} = \frac{1}{2}(a + b)h, \qquad S_{\text{паралелограма}} = ah.$$

21. Клас «Об'ємні тіла», що має властивість (сторона або висота) і методи формування рядка з описом об’єкта і обчислення об’єму. Описати класи-нащадки: циліндр з властивістю (радіус) і куб. Для обчислення об’єму скористатися формулами

$$V_{\text{цилінда}} = \pi R^2 h, \qquad V_{\text{куба}} = a^3.$$

22. Клас «Об'ємні тіла», що має властивість (сторона або висота) і методи формування рядка з описом об’єкта і обчислення площі повної поверхні. Описати класи-нащадки: циліндр з властивістю (радіус) і куб. Для обчислення площі повної поверхні скористатися формулами

$$S_{\text{циліндра}} = 2\pi R(R + h), \qquad S_{\text{куба}} = 6a^2.$$

23. Клас «Об'ємні тіла», що має властивість (сторона або висота) і методи формування рядка з описом об’єкта і обчислення об’єму. Описати класи-нащадки: конус з властивістю (радіус) і куб. Для обчислення об’єму скористатися формулами

$$V_{\text{конуса}} = \frac{1}{3}\pi R^2 h, \qquad V_{\text{куба}} = a^3.$$

24. Клас «Плоскі фігури», що має властивості (сторона 1, висота) і методи формування рядка з описом об’єкта і обчислення площі. Описати класи-нащадки: трикутник і трапеція з властивістю (сторона 2). Для обчислення площ скористатися формулами

$$S_{\text{трикутника}} = \frac{1}{2}ah, \qquad S_{\text{трапеції}} = \frac{1}{2}(a+b)h.$$

25. Клас «Плоскі фігури», що має властивості (сторона 1, сторона 2) і методи формування рядка з описом об’єкта і обчислення площі. Описати класи-нащадки: прямокутник і трапеція з властивістю (висота). Для обчислення площ скористатися формулами

$$S_{\text{прямокутника}} = ab, \qquad S_{\text{трапеції}} = \frac{1}{2}(a+b)h.$$

Контрольні запитання

1. Поясніть поняття класу, поля і методу.
2. Поясніть поняття інкапсуляції.
3. Поясніть поняття поліморфізму.
4. Опишіть основні принципи наслідування.
5. Назвіть правила задання віртуальних методів.
6. Для чого призначено модифікатори, що використовуються в описі класу?

Розрахункова робота

РЕКУРЕНТНЕ СПІВВІДНОШЕННЯ. ВИКОРИСТАННЯ РЕКУРСИВНИХ ФУНКЦІЙ

Мета роботи – вивчити алгоритмізацію і програмування задач з використанням рекурсивних функцій.

Теоретичні відомості

Поняття рекурсії

Рекурсивним називається об'єкт, який частково визначається сам через себе.

Рекурсивні вирази як потужний аналітичний апарат використовують у багатьох областях науки, особливо в математиці.

Розглянемо функцію факторіала n!. Зазвичай її визначають як добуток перших n цілих чисел:

```
n! = 1*2*3*... * n .
```

Такий добуток можна обчислити за допомогою ітеративних конструкцій, наприклад оператора циклу ***for***. Однак існує також інше визначення факторіала, в якому використовується рекурентна формула

```
0!=1 для ∀ n>0 n!=n*(n-1)!
```

Тоді функція для обчислення факторіала має такий вигляд:

```
// рекурсивна функція знаходження факторіала
int factorial(int k)
{
    if(k==1)return 1; return
    k*factorial(k-1);
}
```

Форми рекурсивних функцій

У загальному випадку будь-яка рекурсивна функція `Rec` містить деяку множину операторів *S* і один або кілька операторів рекурсивного виклику. Структура рекурсивної функції може набувати таких форм:

1. З виконанням дій *до рекурсивного виклику* (з виконанням дій на рекурсивному спуску):

```
void f()
{
    S;
    if (умова) f();
}
```

2. З виконанням дій *після рекурсивного виклику* (з виконанням дій на рекурсивному поверненні):

```
void f()
{
    if (умова) f();
    S;
}
```

3. З виконанням дій *як до, так і після рекурсивного виклику* (з виконанням дій як на рекурсивному спуску, так і на рекурсивному поверненні):

```
void f()
{
    S1;
    if (умова) f();
    S2;
}
```

або

```
void f()
{
    if (умова)
    {
        S1;
        f();
        S2;
    }
}
```

Порядок виконання роботи

Розробити програму з віконним інтерфейсом, що обчислює рекурентне співвідношення (табл.17) *за допомогою рекурсії* та контрольний вираз. Значення x ввести з клавіатури ($0 < x < 1$). Виконати завдання двома способами:

1) обчислити суму ряду S_k, якщо кількість членів розкладання дорівнює n. Значення n ввести з клавіатури.
2) виконати обчислення з точністю ε (підсумовувати члени розкладання до тих пір, поки $\varepsilon > |S_k - S_{k-1}|$. Значення ε ввести з клавіатури.

Варіанти завдань до розрахункової роботи

Таблиця 17

Функції для обчислення

Номер варіан-та	Ряд S і контрольний вираз C
1	$S = \frac{3x^2}{4!} - \frac{5x^4}{6!} + \frac{7x^6}{8!} - \cdots + (-1)^{k-1}\frac{(2k+1)x^{2k}}{(2k+2)!} + \cdots$ $C = \frac{1-\cos x}{x^2} - \frac{\sin x}{x} + \frac{1}{2}$
2	$S = \frac{x}{1!} + \frac{x^2}{2!} + \cdots + \frac{x^k}{k!} + \cdots$ $C = e^x - 1$
3	$S = \frac{8\sqrt{x} * x}{3!} + \frac{32\sqrt{x * x}\,x^2}{5!} + \cdots + \frac{8k^2\sqrt{x} * x^k}{(2k+1)!} + \cdots$ $C = (x + 1 - \sqrt{x})e^{\sqrt{x}} - (x + 1 + \sqrt{x})e^{-\sqrt{x}}$
4	$S = \frac{x}{1} + \frac{x^3}{2} + \frac{x^5}{3} + \cdots + \frac{x^{2k-1}}{k} + \cdots$ $C = -\frac{1}{x}\ln(1 - x^2)$
5	$S = \frac{2x^4(5+x)}{5!} + \frac{2x^8(9+x)}{9!} + \cdots + \frac{2x^{4k}(4k+1+x)}{(4k+1)!} + \cdots$ $C = \sin x - 2x - 2 + \cos x + e^x$
6	$S = \frac{x(4-2x+x)}{1*2} + \frac{x^3(8-4x+x)}{3*4} + \cdots + \frac{x^{2k-1}(8k-2kx+x)}{(2k-1)2k} + \cdots$ $C = ln\sqrt{(1+x)^3/(1-x)}$

Продовження табл. 17

Номер варіан-та	Ряд S і контрольний вираз C
7	$S = \frac{(2x)^2}{2!} - \frac{(2x)^4}{4!} + \frac{(2x)^6}{6!} - \cdots + (-1)^{k-1}\frac{(2x)x^{2k}}{(2k)!} + \cdots$ $C = 2sin^2x$
8	$S = \frac{x}{3} - \frac{x^3}{5} + \frac{x^5}{7} - \cdots + (-1)^{k-1}\frac{x^{2k-1}}{2k+1} + \cdots$ $C = \frac{x - arctgx)}{x^2}$
9	$S = \frac{x^4}{3!} + \frac{x^8}{7!} + \frac{x^{12}}{11!} + \cdots + \frac{x^{4k}}{(4k-1)!} + \cdots$ $C = \frac{x}{4}(e^x - 2sinx - e^{-x})$
10	$S = \frac{ln3}{1!} + \frac{ln^2 3}{2!}x^2 + \cdots + \frac{ln^k 3}{k!}x^k + \cdots$ $C = 3^x - 1$
11	$S = \frac{x^2}{4!} - \frac{x^4}{6!} + \frac{x^6}{8!} - \cdots + (-1)^{k-1}\frac{x^{2k}}{(2k+2)!} + \cdots$ $C = \frac{cosx - 1}{x^2} + \frac{1}{2}$
12	$S = \frac{x}{1!} - \frac{x^3}{3!} + \cdots + (-1)^{k-1}\frac{x^{2k-1}}{(2k-1)!} + \cdots$ $C = sinx$
13	$S = \frac{4x^5}{1!} - \frac{x^3}{3!} + \cdots + (-1)^{k-1}\frac{x^{2k-1}}{(2k-1)!} + \cdots$ $C = sinx$

Продовження табл. 17

Номер варіан-та	Ряд S і контрольний вираз C
14	$S=\frac{3x^3}{1!}+\frac{5x^5}{2!}+\frac{7x^7}{3!}+\cdots+\frac{(2k+1)x^{2k+1}}{k!}+\cdots$ $C=(x+2x^3)e^{x^2}-x$
15	$S=\frac{x}{3!}-\frac{x^3}{5!}+\frac{x^5}{7!}-\cdots+(-1)^{k-1}\frac{x^{2k-1}}{(2k+1)!}+\cdots$ $C=\frac{x-sinx}{x^2}$
16	$S=\frac{x-1}{x+1}+\frac{1}{3}\left(\frac{x-1}{x+1}\right)^3+\cdots+\frac{1}{2k-1}\left(\frac{x-1}{x+1}\right)^{2k-1}+\cdots$ $C=\frac{1}{2}lnx$
17	$S=-\frac{x^2}{2!}+\frac{x^4}{4!}-\cdots+(-1)^k\frac{x^{2k}}{(2k)!}+\cdots$ $C=cosx-1$
18	$S=\frac{2x^3}{13}-\frac{2x^5}{15}+\frac{2x^7}{35}-\cdots+(-1)^{k-1}\frac{2x^{2k+1}}{4k^2-1}+\cdots$ $C=(1+x^2)arctgx-x$
19	$S=\frac{3x^2}{2!}-\frac{9x^4}{4!}+\frac{19x^6}{6!}-\cdots+(-1)^{k-1}\frac{(2k^2+1)x^{2k}}{(2k)!}+\cdots$ $C=1+\frac{x}{2}sinx+\left(\frac{x^2}{2}-1\right)cosx$
20	$S=\frac{(x-1)^2}{1}-\frac{(x-1)^4}{2}+\frac{(x-1)^6}{3}-\cdots+(-1)^{k-1}\frac{(x-1)^{2k}}{(2k)!}+\cdots$ $C=\ln(2-2x+x^2)$

Продовження табл. 17

Номер варіан-та	Ряд S і контрольний вираз C
21	$S = \frac{4x}{1!} + \frac{4x^5}{5!} + \frac{4x^9}{9!} + \cdots + \frac{4x^{4k-3}}{(4k-3)!} + \cdots$ $C = e^x - 2sinx - e^{-x}$
22	$S = x - \frac{x^2}{2} + \frac{x^3}{3} - \cdots + (-1)^{k-1}\frac{x^k}{k} + \cdots$ $C = \ln(1+x)$
23	$S = \frac{x^2}{2} - \frac{x^4}{12} + \frac{x^6}{30} - \cdots + (-1)^{k-1}\frac{x^{2k}}{2k(2k-1)} + \cdots$ $C = xarctgx - ln\sqrt{1+x^2}$
24	$S = \frac{x^2}{2!} - \frac{3x^4}{4!} + \frac{5x^6}{6!} - \cdots + (-1)^{k-1}\frac{(2k-1)x^{2k}}{2k!} + \cdots$ $C = cosx + xsinx - 1$
25	$S = -\frac{x^3}{3} + \frac{x^5}{5} - \cdots + (-1)^k\frac{x^{2k+1}}{2k+1} + \cdots$ $C = arctgx - x$
26	$S = \frac{4x}{1!} + \frac{8x^3}{3!} + \frac{12x^5}{5!} + \cdots + \frac{4kx^{2k-1}}{(2k-1)!} + \cdots$ $C = (x+1)e^x - (x-1)e^{-x}$
27	$S = \frac{3x^2}{4!} - \frac{5x^4}{6!} + \frac{7x^6}{8!} - \cdots + (-1)^{k-1}\frac{(2k+1)x^{2k}}{(2k+2)!} + \cdots$ $C = \frac{1-cosx}{x^2} - \frac{sinx}{x} + \frac{1}{2}$

Закінчення табл. 17

Номер варіан-та	Ряд S і контрольний вираз C
28	$S = \frac{x(3+x)}{3!} - \frac{x^3(5+x)}{5!} + \cdots + (-1)^{k-1}\frac{x^{2k-1}(2k+1+x)}{(2k+1)!} + \cdots$ $C = \frac{1 - cosx - sinx}{x} + 1$
29	$S = \frac{2}{3}\frac{x^3}{1+x^2} + \frac{2\cdot 4}{3\cdot 5}\frac{x^5}{(1+x^2)^2} + \cdots + \frac{2\cdot 4\cdot\cdots\cdot 2kx^{2k+1}}{3\cdot 5\cdot\cdots\cdot(2k+1)(1+x^2)^k} + \cdots$ $C = (1+x^2)arctgx - x$
30	$S = x + \frac{x^3}{3} + \frac{x^5}{5} + \frac{x^7}{7} + \cdots + \frac{x^{2k-1}}{2k-1} + \cdots$ $C = \frac{1}{2}ln\frac{1+x}{1-x}$

Контрольні запитання

1. Назвіть особливості виконання операторів: циклів, умовних, введення/виведення.
2. Опишіть поняття функції.
3. Що таке формальні й фактичні параметри?
4. Що таке параметри-значення і параметри-змінні?
5. Що таке глобальні й локальні змінні?
6. Поясніть поняття «область видимості змінних».
7. Поясніть поняття «область існування змінних».
8. Що таке рекурсія?
9. Що таке глибина рекурсії?

БІБЛІОГРАФІЧНИЙ СПИСОК

Албахари, Дж. C# 6.0. Справочник. Полное описание языка : пер. с англ. / Дж. Албахари, Б. Албахари. – М. : Вильямс, 2018. – 1040 с.

Джуст, В. Разработка обслуживаемых программ на языке C# : пер. с англ. / В. Джуст. – М. : ДМК Пресс, 2017. – 192 с.

Нейгел, К. C# 5.0 и платформа .NET 4.5 для профессионалов : пер. с англ. / К. Нейгел, Б. Ивьен. – М. : Вильямс, 2014. – 1440 с.

Рихтер, Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C# : пер. с англ. / Дж. Рихтер. – СПб. : Питер, 2018. – 896 с.

Рихтер, Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C# : пер. с англ. / Дж. Рихтер. – СПб. : Питер, 2018. – 896 с.

Троелсен, Э. Язык программирования C# 7 на платформе .NET и .NET Core : пер. с англ. / Э. Троелсен, Ф. Джепикс. – М. : Вильямс, 2016. – 1328 с.

Шарп, Дж. Microsoft Visual C#. Подробное руководство : пер. с англ. / Дж. Шарп. – СПб. : Питер, 2017. – 848 с.

Навчальне видання

Соколова Євгенія Віталіївна
Лучшева Оксана Вадимівна
Манжос Юрій Семенович

ПРОГРАМУВАННЯ МОВОЮ C#

Редактор В. І. Філатова

Зв. план, 2021
Підписано до видання 30.07.2021
Ум. друк. арк. 4,2. Обл.-вид. арк. 4,69. Електронний ресурс

Видавець і виготовлювач
Національний аерокосмічний університет ім. М. Є. Жуковського
«Харківський авіаційний інститут»
61070, Харків-70, вул. Чкалова, 17
http://www.khai.edu
Видавничий центр «ХАІ»
61070, Харків-70, вул. Чкалова, 17
izdat@khai.edu

Свідоцтво про внесення суб’єкта видавничої справи
до Державного реєстру видавців, виготовлювачів і розповсюджувачів
видавничої продукції сер. ДК № 391 від 30.03.2001